

CFC Informaticien

Travail Pratique Individuel (TPI)

Agora



Candidat	<i>Nardou Thomas</i>
École professionnelle / Entreprise formatrice	ETML
Chef de projet	Jonathan Melly
Expert 1	<i>Nicolas Borboën</i>
Expert 2	<i>Michael Wyssa</i>
Date de début	27.04.2026
Date de fin	28.05.2026
Version du rapport	1.0.177

Table des Matières

1	Partie 1 - Rapport de projet	10
1.1	Contexte et mandat	10
1.2	Méthode et organisation	11
1.3	Analyse des exigences	12
1.4	Gestion des risques et mesures	13
1.5	Conception	14
1.5.1	Maquettes	14
1.5.2	Base de données	19
1.5.3	Architecture	25
1.5.4	Choix techniques	28
1.5.5	Authentification	30
1.5.6	Script (CRON)	31
1.6	Planification et suivi	34
1.7	Réalisation	35
1.7.1	Structure du dépôt	35
1.7.2	Version des outils utilisés	36
1.8	Tests et validation	36
1.8.1	Stratégie de tests	36
1.8.2	Cas de tests	36
1.8.3	Tests manuels	37
1.8.4	Tests unitaires	38
1.8.5	Bugs connus	39
1.9	Livraison	41
1.10	Analyse critique	41
1.10.1	Application	41
1.10.2	API mail vs SMTP	41
1.10.3	Configuration DNS	42
1.11	Conclusion et bilan personnel	45
2	Partie 2 - Documentation du produit	46
2.1	Prérequis	46
2.2	Installation / Déploiement	46
2.2.1	Clonage du repository	46
2.2.2	Installation des dépendances	46
2.2.3	Configurer les variables d'environnement	47
2.2.4	Initialiser la base de données	47
2.2.5	Démarrer l'application	48

2.2.6	Commandes disponibles	48
2.3	Guide d'utilisation	49
2.3.1	Inscription.....	49
2.3.2	Désinscription.....	50
2.3.3	Connexion	51
2.3.4	Création d'une campagne	52
3	Résumé.....	57
3.1	Contexte et mandat.....	57
3.2	Solution réalisée	57
3.3	Résultats principaux	57
3.4	Conclusion.....	57
4	Annexes.....	58
4.1	Intelligence artificielle	58
4.2	Journal de travail	58
4.2.1	Total des heures :	58
4.2.2	Pondération des tâches	58
4.2.3	Explications.....	60
4.2.4	Journal	61
4.3	Sources et références.....	62
4.4	Liste des livrables remis	62

Glossaire

Terme	Définition
A	
API REST	Une API REST (Representational State Transfer) est une interface de programmation web qui respecte les contraintes de l'architecture REST. Elle permet à deux systèmes de communiquer via HTTP en exposant des endpoints qui renvoient des données structurées (généralement au format JSON). Les méthodes standard sont GET, POST, PUT, PATCH et DELETE.
Asynchrone	En programmation, une opération asynchrone est une tâche qui s'exécute en arrière-plan sans bloquer le fil principal du programme. Le code continue de s'exécuter et reçoit le résultat de l'opération plus tard, via un callback, une promesse (Promise) ou async/await. C'est essentiel pour les opérations longues comme les requêtes HTTP ou les envois d'e-mails.
B	
Boolean	Un booléen dans MariaDB permet de stocker une valeur de vérité logique Vrai ou Faux. Techniquement, MariaDB ne possède pas de type de données booléen natif. Le type BOOLEAN (ou BOOL) y est simplement un synonyme de TINYINT(1).
Bounce messages	Les messages de rebond (bounce messages) SMTP sont des notifications automatiques (NDR : Non-Delivery Report) envoyées par un serveur de messagerie pour informer l'expéditeur qu'un e-mail n'a pas pu être remis. Ils sont classés en deux types : • Soft Bounce : Indique que le problème est passager (serveur destinataire hors ligne, filtre anti-spam strict, etc). Dans ce cas-là, le serveur tentera de renvoyer l'e-mail plus tard. • Hard Bounce : Indique que le problème est irréversible (l'adresse e-mail n'existe pas ou le nom de domaine du destinataire est invalide).
C	
Cache	Un cache est un mécanisme de stockage temporaire qui conserve une copie des données fréquemment consultées afin d'accélérer les accès ultérieurs. Dans une application web, le cache peut intervenir côté serveur (Redis, Memcached) ou côté client (navigateur). Il réduit la charge sur la base de données et améliore les performances globales.
CORS	CORS (<i>Cross-Origin Resource Sharing</i>) est un mécanisme de sécurité HTTP qui contrôle la façon dont les ressources d'un serveur web peuvent être demandées depuis un autre domaine. Il utilise des en-têtes HTTP (<i>Access-Control-Allow-Origin</i> , etc.) pour indiquer aux navigateurs les origines autorisées à accéder aux ressources d'une API.
D	
DATETIME	Le type de données DATETIME est utilisé pour stocker des informations combinant à la fois une date et une heure. Il est idéal pour enregistrer des événements précis comme le moment exact d'une commande ou la création d'un compte. Celui-ci est enregistré sous la forme " AAAA-MM-JJ HH:MM:SS " et est soumis au fuseau horaire du serveur.
DMARC	DMARC (Domain-based Message Authentication, Reporting, and Conformance) est un protocole de sécurité e-mail publié dans le DNS sous forme d'enregistrement TXT. Il permet d'authentifier les e-mails, d'empêcher l'usurpation de domaine (phishing) et de demander aux serveurs de messagerie de rejeter ou mettre en quarantaine les e-mails frauduleux qui échouent aux vérifications.
DNS	Le DNS (Domain Name System ou Système de noms de domaine) est l'annuaire du web. Son rôle principal est de traduire un nom de domaine compréhensible par un humain (ex: google.com) en une adresse IP (par ex : 173.194.39.78).

Terme	Définition
E	
Endpoint	Un endpoint (point de terminaison) est une adresse spécifique exposée par une API à laquelle un client peut envoyer une requête HTTP pour effectuer une action ou récupérer des données. Par exemple, "GET /api/users" est un endpoint permettant d'accéder aux ressources utilisateur. Chaque endpoint est associé à une méthode HTTP (GET, POST, PUT, DELETE, PATCH).
F	
Foreign key	Une clé étrangère (<i>foreign key</i>) dans MariaDB est une contrainte qui garantit l'intégrité référentielle entre deux tables. Elle permet de lier une colonne (ou un groupe de colonnes) d'une table.
H	
HTTPS	HTTPS (<i>HyperText Transfer Protocol Secure</i>) est la version sécurisée du protocole HTTP. Il chiffre les données échangées entre le navigateur et le serveur grâce au protocole TLS (Transport Layer Security), garantissant la confidentialité, l'intégrité des données et l'authenticité du serveur via un certificat SSL/TLS.
I	
Injection SQL	Une injection SQL est une faille de sécurité qui permet à un attaquant d'insérer ou de manipuler des requêtes SQL en exploitant des entrées utilisateur non filtrées. Elle peut permettre l'accès, la manipulation de données en base. La prévention passe par l'utilisation de requêtes préparées (<i>prepared statements</i>) et la validation des entrées.
Integer	INTEGER (ou INT) est le type de données standard utilisé pour stocker des nombres entiers (sans décimale). Il occupe 4 octets (32 bits) de mémoire et permet de stocker des valeurs comprises entre -2 147 483 648 et 2 147 483 647. Il est idéal pour les identifiants ou les quantités. Il existe cependant d'autres types de INT : • TINYINT : encodé sur 8 bits celui-ci peut aller de -128 à 127 • SMALLINT : encodé sur 16 bits celui-ci peut aller de -32 768 à 32 767 • MEDIUMINT : encodé sur 24 bits celui-ci peut aller de -8 388 608 à 8 388 607 • BIGINT : encodé sur 64 bits celui-ci peut aller de -9 223 372 036 854 775 808 à 9 223 372 036 854 775 807
L	
LongText	Le type LONGTEXT dans MariaDB est un type de données utilisé pour stocker de très longs textes. Il peut contenir jusqu'à 4 Go (4 294 967 295 caractères) de texte par enregistrement, ce qui est idéal pour stocker le contenu d'articles, des commentaires longs, ou des données au format JSON.
LPD	LPD signifie principalement Loi fédérale sur la protection des données en Suisse (souvent nLPD pour la nouvelle version du 1er septembre 2023), visant à protéger la vie privée des personnes lors du traitement de leurs données. Elle s'applique aux entreprises et personnes privées.

Terme	Définition
M	
MariaDB	MariaDB est un système de gestion de bases de données relationnelles (SGBDR) open source très populaire. Créé par les développeurs originaux de MySQL en 2009, il s'agit d'une alternative directe conçue pour rester libre, performante et entièrement compatible avec ce dernier.
Maily	Maily est une application web moderne pour envoyer et gérer des e-mails avec une interface intuitive de rédaction et de gestion des modèles. Développée dans le cadre du P-APPRO2 l'application utilise une API publique créée spécifiquement pour l'occasion permettant l'envoi d'e-mails.
Middleware	Un middleware est une couche logicielle intermédiaire placée entre la requête HTTP entrante et le gestionnaire de route final. Dans une API, il intervient pour traiter, valider ou transformer les requêtes (authentification JWT, journalisation, gestion des erreurs, limitation de débit). Il suit le principe d'une chaîne de responsabilité (pipeline).
MVC	Le Modèle-Vue-Contrôleur (MVC) est un motif d'architecture logicielle qui sépare une application en trois composants distincts : le Modèle (données), la Vue (interface utilisateur) et le Contrôleur (logique de contrôle). Cette structure facilite la maintenance, le développement collaboratif et la réutilisation du code.
N	
Next JS	Next.js est un framework React open-source populaire, développé par Vercel, conçu pour créer des applications web. Il permet le rendu côté serveur (SSR) et la génération de sites statiques (SSG) par défaut, améliorant ainsi les performances et le référencement naturel.
O	
ORM	Un ORM (<i>Object-Relational Mapping</i>) est une technique qui permet d'interagir avec une base de données relationnelle en utilisant un langage de programmation orienté objet, sans écrire de SQL brut. Il fait correspondre les tables de la base à des classes et les lignes à des objets. Des exemples populaires incluent Prisma (TypeScript), Doctrine (PHP) ou Sequelize (Node.js).
P	
PHP	PHP (officiellement PHP : <i>Hypertext Preprocessor</i>) est un langage de script open source exécuté côté serveur, principalement utilisé pour créer des sites web et des applications dynamiques. Contrairement au HTML qui est statique, PHP traite les données sur le serveur (connexion aux bases de données, formulaires, etc.).
Primary key	Une clé primaire (<i>Primary Key</i>) en SQL est une contrainte qui permet d'identifier de manière unique chaque ligne (enregistrement) dans une table. Elle agit comme un identifiant exclusif et unique, garantissant qu'aucune ligne ne peut avoir les mêmes données pour cette colonne.

Terme	Définition
R	
React	React est une bibliothèque JavaScript open-source, développée par Meta (Facebook), pour construire des interfaces utilisateur (UI) interactives. Elle repose sur le concept de composants réutilisables et d'un DOM virtuel (Virtual DOM) pour optimiser les mises à jour de l'interface. React est la base de frameworks comme Next.js.
RGPD	Le RGPD (Règlement Général sur la Protection des Données) est le texte de référence européen entré en vigueur en 2018 qui encadre le traitement des données personnelles. Il renforce le contrôle des citoyens sur leurs données et harmonise les règles au sein de l'UE pour les entreprises publiques et privées.
S	
SMTP	Le SMTP (Simple Mail Transfer Protocol) est la norme internet utilisée pour envoyer des e-mails entre serveurs. Il agit comme un facteur numérique qui prend votre message depuis votre application (client mail) et le transporte vers le serveur de messagerie du destinataire.
SPF	SPF (Sender Policy Framework) est un protocole d'authentification e-mail publié dans le DNS sous forme d'enregistrement TXT. Il permet au propriétaire d'un domaine de spécifier les serveurs de messagerie autorisés à envoyer des e-mails en son nom, contribuant ainsi à lutter contre l'usurpation d'identité (spoofing) et le phishing, en complément de DKIM et DMARC.
SSR	En informatique, SSR signifie <i>Server-Side Rendering</i> (rendu côté serveur). Il s'agit d'une technique de développement web permettant de générer le contenu HTML d'une page directement sur le serveur plutôt que dans le navigateur de l'utilisateur (le client).
String	String (chaîne de caractères) désigne les types de données utilisés pour stocker du texte ou des données binaires. C'est une composante essentielle d'une base de données relationnelle, pour stocker des noms, adresses, e-mails, etc.
T	
Tests unitaires	Les tests unitaires sont des procédures automatisées, rédigées par les développeurs, qui vérifient le bon fonctionnement d'une portion précise et isolée d'un programme (fonction, méthode, classe). Ils garantissent qu'une unité de code produit le résultat attendu pour des entrées données, assurant la fiabilité du code et facilitant la maintenance.
Token JWT	Un JWT (<i>JSON Web Token</i>) est un format standard sécurisé (RFC 7519) permettant de transmettre des informations signées entre plusieurs parties : les deux premières sont sous la forme d'un objet JSON et la troisième est la signature de la clé. Il est principalement utilisé pour l'authentification et la vérification des autorisations dans les applications web et les API.
TypeScript	TypeScript est un langage de programmation sur-ensemble de JavaScript (superset) développé par Microsoft. Il ajoute un typage statique optionnel à JavaScript, ce qui signifie qu'il est possible de définir le type de données (chaîne, nombre, objet) attendu dans votre code. Il améliore la fiabilité du code en détectant les erreurs de type avant l'exécution.

Terme	Définition
W	
Waterfall	La méthode de gestion de projet Waterfall (en cascade) est une approche linéaire et séquentielle où chaque phase doit être finalisée avant de passer à la suivante. Idéal pour des projets aux exigences fixes et claires, ce modèle rigoureux planifie tout dès le départ, facilitant la gestion budgétaire et les délais.

1 Partie 1 - Rapport de projet

1.1 Contexte et mandat

Le projet s'inscrit dans le cadre du Travail Pratique Individuel (TPI) de fin de formation. Le mandat a été défini et approuvé par le chef de projet Jonathan Melly ainsi que par les experts. Il couvre une période de réalisation allant du 27 avril 2026 au 28 mai 2026 pour un volume de travail estimé à 90h.

Le périmètre du projet englobe la conception et le développement d'Agora, une application web de gestion et de diffusion de newsletters. Cette plateforme doit permettre à un administrateur de créer, planifier et suivre des campagnes d'envoi, tout en garantissant une diffusion fiable, progressive et conforme aux exigences légales en matière de protection des données.

Le projet repose sur une architecture web moderne basée sur JavaScript et Node.js, avec une base de données MariaDB. L'application s'interfacera avec une API d'envoi d'e-mails existante, utilisant la fonction native mail() de PHP. L'envoi des campagnes devra obligatoirement s'effectuer de manière asynchrone via un système de file d'attente et un processus de relais, afin d'assurer la robustesse du système et la maîtrise des envois en masse.

Le projet poursuit plusieurs objectifs principaux, mesurables et datés. L'application devra permettre l'authentification sécurisée des administrateurs ainsi qu'une gestion stricte des rôles selon le principe du moindre privilège. Elle devra offrir un formulaire public d'inscription conforme aux exigences de la LPD, incluant le consentement explicite des abonnés, ainsi qu'un mécanisme de désinscription reposant sur des tokens uniques et sécurisés.

L'outil devra également permettre la création de campagnes en format Markdown, leur planification, leur mise en file d'attente et leur envoi par lots. Chaque campagne suivra un cycle de vie clairement défini : brouillon, planifiée, en cours, terminée ou échouée. Un système de suivi devra mesurer les performances des campagnes grâce au tracking des ouvertures via un pixel invisible et au suivi des clics par réécriture dynamique des liens.

L'API REST et l'application devront garantir un haut niveau de sécurité. Les données personnelles devront être protégées, les tokens invalides ou expirés rejetés, et aucune vulnérabilité critique, notamment liée aux injections ou aux accès non autorisés, ne devra être présente. Une attention particulière sera portée à la conformité légale, à la protection des données et à la justification des mesures de sécurité mises en œuvre.

Le candidat s'engage à fournir l'ensemble des livrables suivants au chef de projet et aux deux experts à l'échéance du TPI. Le code source complet sera déposé sur un dépôt Git distant et versionné par des commits réguliers. Il sera accompagné d'une documentation d'installation, de configuration et d'utilisation.

1.2 Méthode et organisation

Pour la gestion de ce projet, il a été décidé d'utiliser la méthode "*Waterfall*". Ce choix se justifie avant tout parce que le cahier des charges a été remis de façon complète et définitive dès le premier jour sans possibilité de modification en cours de réalisation. Les fonctionnalités attendues, les contraintes techniques et les livrables étaient donc clairement définis et stables dès le départ, ce qui correspond précisément au contexte dans lequel la méthode Waterfall s'applique le mieux.

L'application est versionnée sur GitHub pour plusieurs raisons. La première concerne la sauvegarde du code et de la documentation, ce qui évite de dépendre d'un seul ordinateur. La seconde, et la principale, est la traçabilité car elle permet de suivre l'évolution du projet et de revenir à une version antérieure à tout moment.

Bien que cela ne constitue pas une pratique recommandée, certains fichiers binaires, tels que le rapport et le journal de travail, ont également été intégrés au dépôt. En effet, les systèmes de gestion de versions sont avant tout conçus pour les fichiers texte, sur lesquels ils peuvent suivre précisément les modifications. Toutefois, dans le cadre de ce projet, ce choix permet de centraliser l'ensemble des livrables au même endroit et d'en assurer la sauvegarde ainsi que l'historique.

Pour la planification, un diagramme de Gantt a été utilisé, car il offre une représentation visuelle claire de l'ensemble des tâches à réaliser, de leur durée ainsi que de leur enchaînement dans le temps. Il permet de structurer le projet en différentes phases, de définir des jalons importants et d'anticiper les dépendances entre les activités.

1.3 Analyse des exigences

Catégorie	Exigence	Description
Fonctionnelle	Authentification	Permettre à un administrateur de se connecter de manière sécurisée à l'application.
Fonctionnelle	Gestion des rôles	Distinguer les droits entre administrateur et utilisateur public selon le principe du moindre privilège.
Fonctionnelle	Gestion des abonnements	Offrir un formulaire public d'inscription avec consentement explicite et un mécanisme de désinscription automatisé.
Fonctionnelle	Gestion des campagnes	Permettre la création, la planification, la modification et le suivi des campagnes de newsletters.
Fonctionnelle	Édition du contenu	Fournir un éditeur simple prenant en charge le format Markdown pour la rédaction des newsletters.
Fonctionnelle	Gestion des contacts	Consulter la liste des abonnés actifs et permettre la suppression manuelle d'un contact.
Fonctionnelle	Envoi asynchrone	Traiter les campagnes via une file d'attente afin d'assurer un envoi progressif et fiable.
Fonctionnelle	Tracking	Mesurer les ouvertures, les clics, les inscriptions et les désinscriptions.
Fonctionnelle	Tableau de bord	Visualiser les indicateurs clés de performance des campagnes.
Fonctionnelle	Gestion des erreurs	Détecter les échecs d'envoi, enregistrer les erreurs et permettre la reprise automatique ou manuelle.
Non fonctionnelle	Performance	Traiter les envois par lots de manière fluide et sans surcharge du système ou de l'API d'envoi.
Non fonctionnelle	Sécurité	Protéger les données personnelles, sécuriser les tokens de désinscription et contrôler les accès.
Non fonctionnelle	Conformité légale	Respecter les exigences de la LPD et, par extension, les principes du RGPD.
Non fonctionnelle	Fiabilité	Garantir l'intégrité des envois et la reprise en cas d'échec partiel d'un lot.
Non fonctionnelle	Maintenabilité	Concevoir une architecture modulaire, documentée et facilement évolutive.

Catégorie	Exigence	Description
Non fonctionnelle	Ergonomie (UX)	Proposer une interface claire, intuitive et adaptée aux besoins des administrateurs.
Non fonctionnelle	Compatibilité	Assurer le bon fonctionnement sur les navigateurs web modernes.
Non fonctionnelle	Traçabilité	Journaliser les opérations importantes afin de faciliter le suivi, l'audit et le débogage.

1.4 Gestion des risques et mesures

Plusieurs risques ont été identifiés. Par exemple une découverte tardive de failles de sécurité, notamment au niveau de l'authentification ou de la validation des données, nécessiterait des corrections importantes en fin de projet et avoir un impact sur le planning. Certaines tâches pourraient s'avérer plus complexes qu'initialement. Un autre risque serait tout simplement des absences ou des retards qui ralentiraient la progression du projet. Enfin, la disponibilité limitée du chef de projet ou des experts peut ralentir la prise de décision et la validation de certains choix techniques/fonctionnels ou simplement des réponses à des questions bloquantes.

1.5 Conception

1.5.1 Maquettes

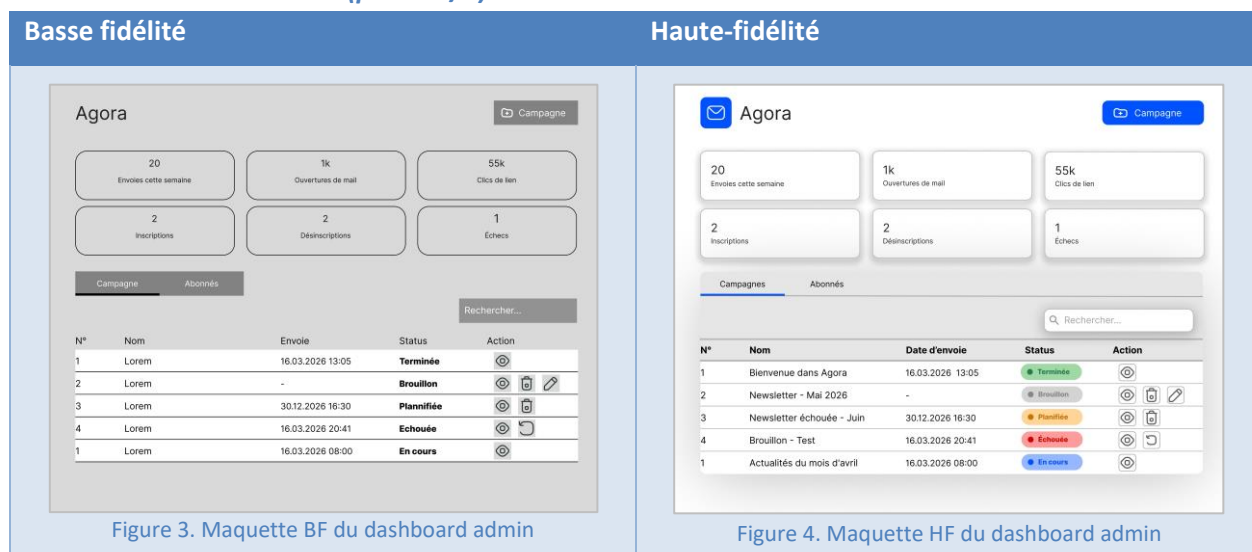
Les maquettes ont été réalisées, d'abord en basse fidélité (BF), puis en haute-fidélité (HF), ce qui a permis de concevoir et de valider progressivement l'interface utilisateur, en passant d'une représentation fonctionnelle centrée sur la structure et l'ergonomie à une version visuelle proche du produit final.

Aucune palette de couleurs spécifique n'a été utilisée. L'idée dernière est de rester simple tout en étant intuitif. Les boutons sont bleus pour pouvoir faire un contraste avec le fond blanc et si plusieurs boutons sont l'un à côté de l'autre, d'autres couleurs (comme le vert et le gris) ont été utilisées afin de les différencier.

1.5.1.1 Login admin



1.5.1.2 Dashboard admin (partie 1/2)



1.5.1.3 Dashboard admin (partie 2/2)

Basse fidélité

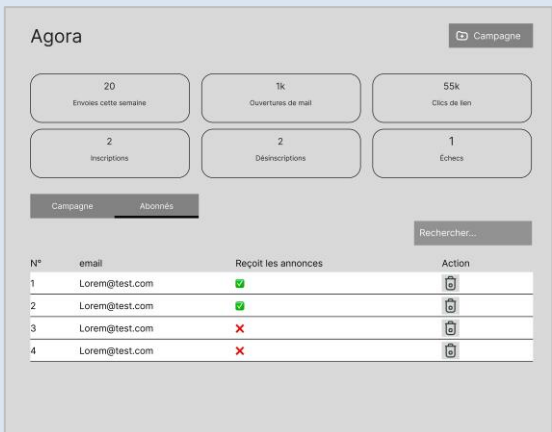


Figure 5. Maquette BF du dashboard admin

Haute-fidélité

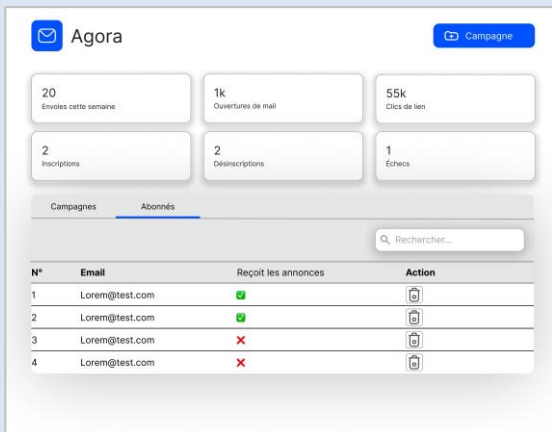


Figure 6. Maquette HF du dashboard admin

1.5.1.4 Création d'une campagne

Basse fidélité



Figure 7. Maquette BF de la page de création d'une campagne

Haute-fidélité



Figure 8. Maquette HF de la page de création d'une campagne

1.5.1.5 Planification d'une campagne

Basse fidélité

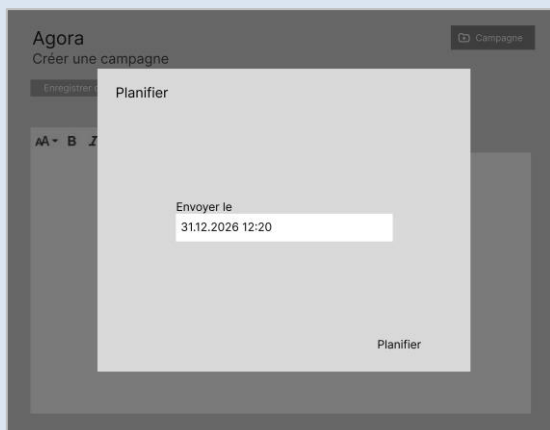


Figure 9. Maquette BF du popup de planification d'une campagne

Haute-fidélité

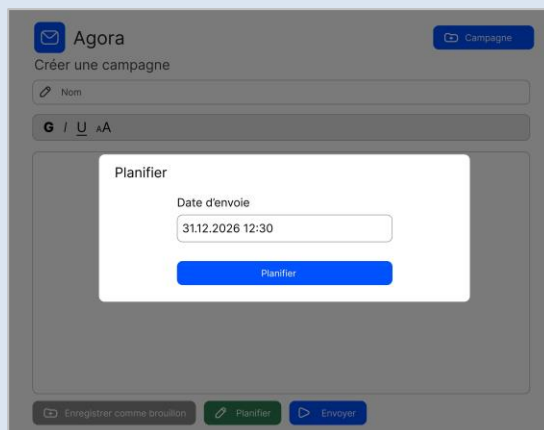


Figure 10. Maquette HF du popup de planification d'une campagne

1.5.1.6 Choix des contacts

Basse fidélité

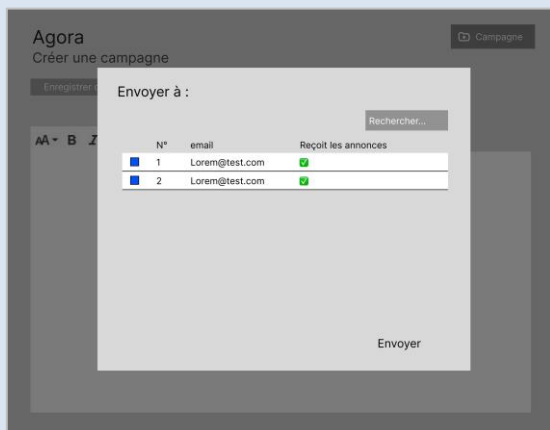


Figure 11. Maquette BF de la popup de choix de contact

Haute-fidélité

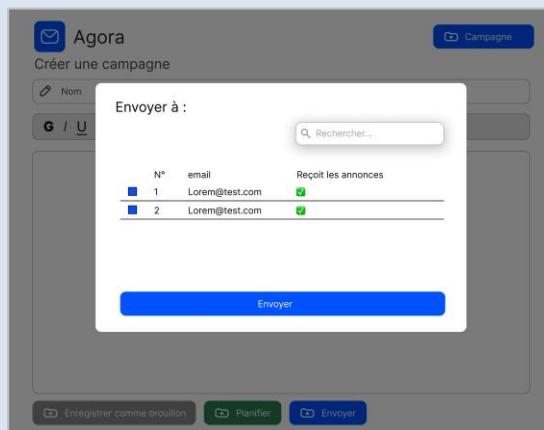


Figure 12. Maquette HF du popup de choix de contact

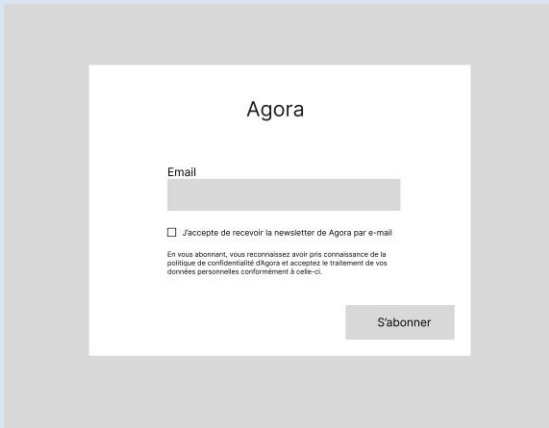
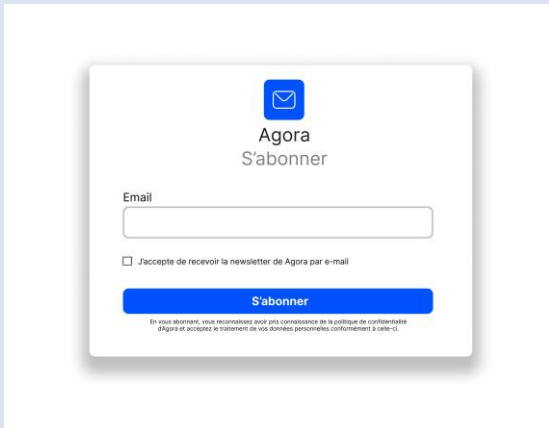
1.5.1.7 Détails d'une campagne (partie 1/2)



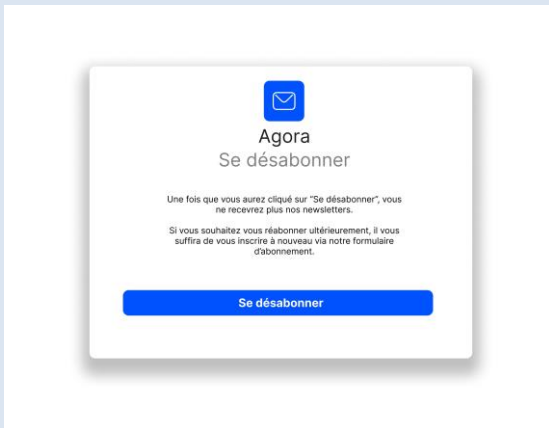
1.5.1.8 Détails d'une campagne (partie 2/2)



1.5.1.9 Page d'abonnement

Basse fidélité	Haute-fidélité
 Figure 17. Maquette BF de la page de souscription	 Figure 18. Maquette HF de la page de souscription

1.5.1.10 Page de désabonnement

Basse fidélité	Haute-fidélité
 Figure 19. Maquette BF de la page de désabonnement	 Figure 20. Maquette HF de la page de désabonnement

1.5.2 Base de données



Figure 21. Schéma de la base de données

1.5.2.1 Modèle de donnée

t_admins : Table des administrateurs de l'application. Ils sont caractérisés par : un prénom et un nom, un e-mail et un mot de passe hashé.

t_sessions : Cette table contient les sessions (cache) de connexion de chaque administrateur. Elles sont caractérisées par le token de l'administrateur et l'id de l'administrateur à laquelle elle est rattachée. Cette table possède une relation "one-to-many" avec la table "**t_admin**", car un administrateur peut avoir plusieurs sessions alors qu'une session ne peut être assignée qu'à un seul administrateur.

t_newsLetters : Cette table contient la liste des newsletters. Elles sont caractérisées par : un nom, un body qui correspond au mail qui sera envoyé aux abonnés, la date à laquelle la campagne va être envoyée et son statut. La table possède une relation "one-to-many" avec la table "**t_newsLetters_status**" car une campagne possède un seul et unique statut et qu'un statut peut être utilisé par plusieurs campagnes. Une relation supplémentaire "one-to-many" avec la table "**t_admin**" aurait pu être envisagée mais n'a pas été mise en place car une campagne n'appartient pas à un administrateur spécifique et dans ce cas-là, une relation n'a pas grand intérêt.

t_newLetters_status : Cette table correspond à la liste des différents statuts qu'une campagne peut avoir. Ils sont caractérisés par : le nom d'affichage, la valeur du statut, la couleur du texte, ainsi que la couleur de fond.

t_readers : Cette table correspond à la liste des lecteurs qui sont abonnés aux newsletters. Ils sont caractérisés par : un e-mail, s'ils donnent leur consentement pour recevoir des mails, une date à laquelle ils ont consenti, leur token de désinscription. La table possède une relation "many-to-many" (table de pivot: "**t_newsLetters_readers**") avec la table "**t_newsLetters**" car une campagne peut être envoyée à plusieurs lecteurs et un lecteur peut être assigné à plusieurs campagnes.

t_newsLetters_readers : Table de pivot permettant la relation entre la table "**t_readers**" et "**t_newsLetters**". Elle est caractérisée par la date à laquelle le mail a été envoyé ainsi que la date d'échec. Dans cette table, tous les envois qui ont ou qui vont être envoyés sont enregistrés.

t_emails_clicked/opened : Tables contenant la liste de toutes les interactions faites sur un mail qui a été envoyé à un lecteur. Une interaction est caractérisée par : la campagne à laquelle elle est rattachée, ainsi que la date à laquelle l'interaction a été faite. Les tables possèdent une relation "one-to-many" avec la table "**t_newsLetters**" car une interaction appartient à une campagne alors qu'une campagne peut avoir plusieurs interactions.

1.5.2.2 Dictionnaire de données

Table admin :

Champ	Type	Clé	Description
admin_id	Integer	Primary key	Identifiant de l'administrateur
email	String	Unique	Adresse e-mail de l'administrateur
name	String	-	Prénom de l'administrateur
lastName	String	-	Nom de famille de l'administrateur
password	String	-	Mot de passe hashé
createdAt	DateTime	-	Date et heure de création du compte
updatedAt	DateTime	-	Date et heure de la dernière modification

Table lecteur :

Champ	Type	Clé	Description
reader_id	Integer	Primary key	Identifiant du lecteur
email	String	Unique	Adresse e-mail de l'abonné
consentGiven	Boolean	-	Indicateur du consentement RGPD donné (par défaut: false)
consentGivenAt	DateTime	-	Date et heure du consentement (null si non donné)
unsubscribeToken	String	Unique	Token unique pour le désabonnement
createdAt	DateTime	-	Date et heure de l'inscription
updatedAt	DateTime	-	Date et heure de la dernière modification

Table newsletters :

Champ	Type	Clé	Description
newsLetter_id	Integer	Primary key	Identifiant de la newsletter
name	String	-	Titre/Nom de la newsletter
body	LongText	-	Contenu de la newsletter
sendAt	DateTime	-	Date et heure prévue d'envoi (null si non programmé)
newsLetter_status_fk	Integer	Foreign key	Référence à l'état de la newsletter
createdAt	DateTime	-	Date et heure de création
updatedAt	DateTime	-	Date et heure de la dernière modification

Table newsletters status :

Champ	Type	Clé	Description
status_id	Integer	Primary key	Identifiant du statut
displayName	String	-	Nom affiché du statut (ex : "En attente d'envoi")
status	String	-	Code technique du statut (ex : "pending", "sent", "draft")
textColor	String	-	Couleur du texte pour l'affichage
backgroundColor	String	-	Couleur de fond pour l'affichage
createdAt	DateTime	-	Date et heure de création
updatedAt	DateTime	-	Date et heure de la dernière modification

Table newsletters readers :

Champ	Type	Clé	Description
newsLetter_fk	Integer	Foreign key	Référence à la newsletter
reader_fk	Integer	Foreign key	Référence au lecteur
sentAt	DateTime	-	Date et heure d'envoi de la newsletter au lecteur (null si non envoyé)
failedAt	DateTime	-	Champ pour indiquer que l'envoi a échoué, pour éviter que le lecteur soit traité plusieurs fois

Table sessions :

Champ	Type	Clé	Description
session_id	Integer	Primary key	Identifiant de la session
admin_fk	Integer	Foreign key	Référence à l'administrateur
token	String	Unique	Token JWT de la session
createdAt	DateTime	-	Date et heure de création de la session
updatedAt	DateTime	-	Date et heure de la dernière modification

Table email opened

Champ	Type	Clé	Description
email_opened_id	Integer	Primary key	Identifiant unique auto-incrémenté de l'enregistrement
newsLetter_fk	Integer	Foreign key	Référence à la newsletter ouverte
openedAt	DateTime	-	Date et heure d'ouverture du mail

Table email clicked

Champ	Type	Clé	Description
email_clicked_id	Integer	Primary key	Identifiant unique auto-incrémenté de l'enregistrement
newsLetter_fk	Integer	Foreign key	Référence à la newsletter contenant le lien cliqué
clickedAt	DateTime	-	Date et heure du clic sur le lien

1.5.2.3 Conformité légale

Le système de newsletters implémente plusieurs mécanismes permettant de respecter certaines exigences du RGPD et des règles liées à la protection des données.

1. Respect du consentement (RGPD Art. 7)

Le système applique un filtrage par consentement afin que seuls les lecteurs ayant "`consentGiven: true`" puissent recevoir une newsletter. Cette vérification est réalisée directement au niveau des requêtes Prisma vers la base de données.

2. Droit d'opposition (RGPD Art. 21)

Chaque e-mail envoyé contient un lien de désinscription personnalisé intégrant un token unique associé au lecteur. Ce mécanisme permet une désinscription sécurisée et individualisée.

3. Suivi et tracking (ePrivacy Directive)

Le système intègre des mécanismes de suivi des interactions des utilisateurs tout en garantissant qu'aucune donnée ne permette de remonter à leur identité.

- Tracking des clics via des URLs de suivi intégrées dans les liens
- Tracking d'ouverture via un pixel de suivi inclus dans les e-mails.

4. Protection des données (RGPD Art. 32)

Plusieurs mesures de sécurité sont mises en place afin de protéger les données transmises :

- Authentification du service mail via le token "MAIL_TOKEN"
- Communication sécurisée via HTTPS avec le service Maily
- Limitation des données envoyées aux seules informations nécessaires ("email", "nom", "token")

5. Responsabilité et traçabilité (RGPD Art. 5)

Le système enregistre les informations liées aux envois de newsletters afin d'assurer une traçabilité des opérations :

- Logs d'envoi enregistrés en base de données
- Suivi des statuts des newsletters
- Horodatage des créations, mises à jour et envois.

1.5.2.4 Classification des données

Donnée	Sensibilité	Mesure de protection
E-mail abonné	Sensible	Accès restreint, pas exposé publiquement
unsubscribeToken	Sensible	Unique, généré aléatoirement
Mot de passe admin	Critique	Hashé en base de données
Token de session	Sensible	Expiration, stocké haché
Contenu campagne	Interne	Aucune restriction particulière
Statistiques agrégées	Interne	Aucune restriction particulière

1.5.2.5 Rôles

Au cours de l'analyse, deux rôles ont été sélectionnés :

Nom	Permission(s)
Administrateur	Plein pouvoir sur l'application
Lecteur	S'abonner, se désabonner

En limitant le nombre de rôles, on simplifie la gestion des droits tout en répondant aux besoins fonctionnels (principe du moindre privilège).

1.5.3 Architecture

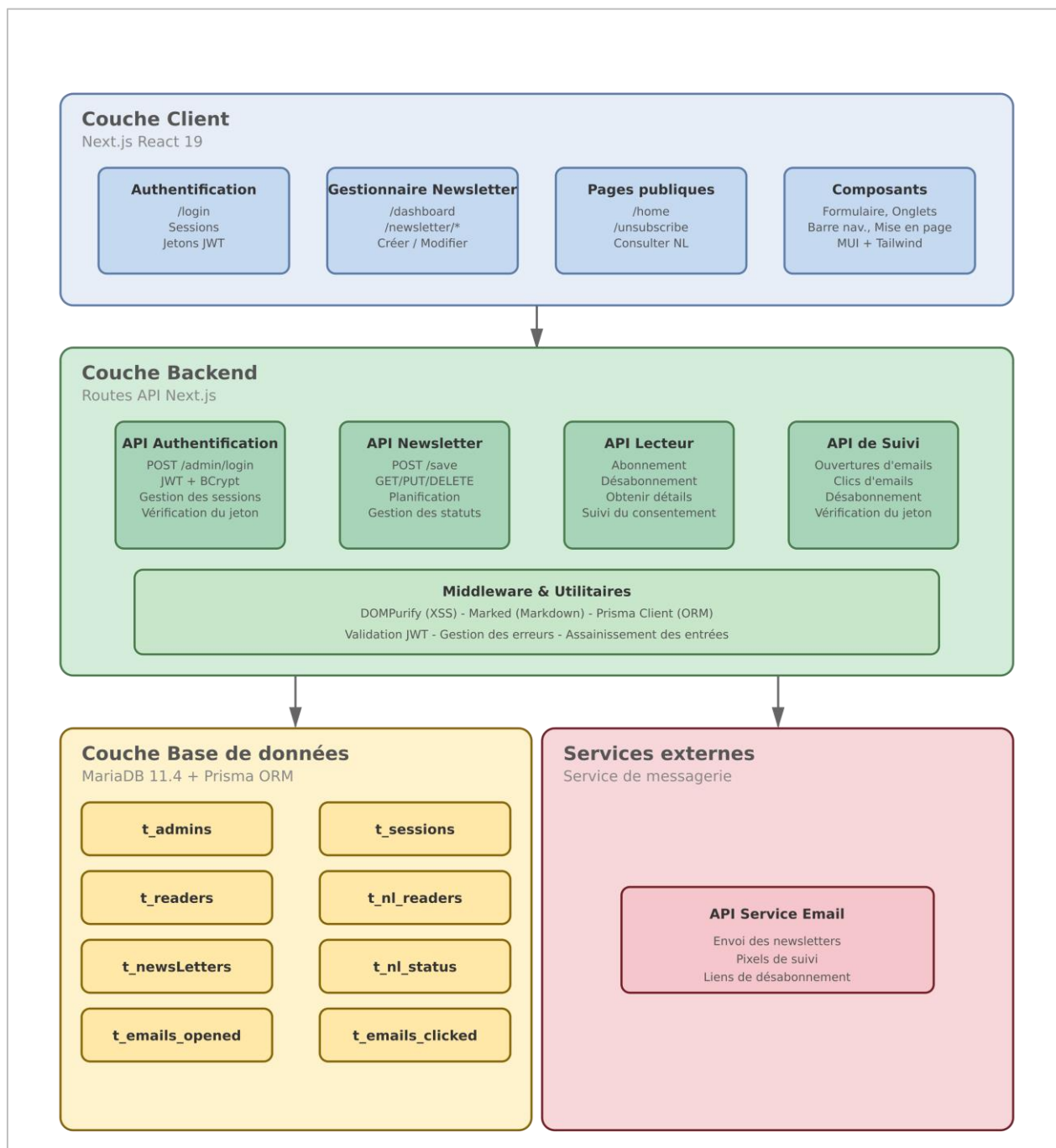


Figure 22. Schéma de l'architecture de l'application

1.5.3.1 Structure du projet

Dossiers principaux:

Dossier	Rôle
/app	Cœur de l'application Next.js contenant toutes les pages, composants et routes API
/app/api	Routes API backend (endpoints REST pour auth, newsletters, readers, tracking)
/app/components	Composants réutilisables (formulaires, navbar, e-mails, tabs)
/app/dashboard	Page d'administration du tableau de bord pour gérer les newsletters et lecteurs
/app/login	Page de connexion et authentification des administrateurs
/app/newsletter	Pages pour visualiser et gérer les newsletters
/app/unsubscribe	Page de désabonnement des lecteurs
/app/privacyPolicy	Page de politique de confidentialité
/lib	Utilitaires et logiques métier réutilisables (queue de newsletters, config Prisma)
/prisma	Configuration de la base de données ORM (schéma, seed)
/public	Assets statiques (images SVG, icônes)
/Doc	Documentation du projet (planification, maquettes, rapports, JDT)
/__test__	Tests unitaires
/generated	Code généré automatiquement (types Prisma)

Fichiers principaux :

Fichier	Rôle
package.json	Dépendances du projet et scripts NPM
tsconfig.json	Configuration TypeScript
next.config.ts	Configuration Next.js
tailwind.config.ts	Configuration Tailwind CSS pour les styles
postcss.config.mjs	Configuration PostCSS pour le traitement CSS
.env	Variables d'environnement (URLs, secrets, clés API)
.env.example	Template des variables d'environnement
eslint.config.mjs	Configuration ESLint pour la qualité du code
docker-compose.yml	Configuration Docker pour MariaDB
instrumentation.ts	Fichier permettant le lancement du script d'envoi au démarrage de l'application
proxy.ts	Middleware afin de rendre le login obligatoire pour certaines routes
prisma.config.ts	Configuration Prisma
prisma/schema.prisma	Schéma de base de données (tables, relations)
prisma/seed.ts	Script de peuplement initial de la base de données
lib/newsletter-queue.ts	Gestion de la queue d'envoi des newsletters
lib/prisma.ts	Instance Prisma Client centralisée
app/layout.tsx	Layout racine Next.js (structure HTML générale)
app/layout-client.tsx	Layout côté client
app/page.tsx	Page d'accueil (page d'inscription)
app/dashboard/page.tsx	Page dashboard administrateur
app/login/page.tsx	Page de connexion
app/components/newsLetterForm.tsx	Formulaire de création/édition de newsletter
app/components/newsLettersTab.tsx	Onglet de la liste des newsletters
app/components/newsLetterForm.tsx	Formulaire de création/édition de newsletter
app/components/newsLettersTab.tsx	Onglet gestion des newsletters
app/components/readersTab.tsx	Onglet de la liste des lecteurs
app/components/navbar.tsx	Barre de navigation
app/components/EmailHeader.tsx	En-tête des e-mails
app/components/EmailFooter.tsx	Pied de page des e-mails

1.5.4 Choix techniques

Nom	Technologie
Frontend	Next JS
Backend	Next JS
Stack	TypeScript
Base de données	MariaDB (imposé)

1.5.4.1 Justification

Nom	Poids	Avantages	Inconvénient
Architecture	8/10	Le frontend et le backend en un seul projet	L'arborescence devient plus conséquente
Rendu	3/10	Utilisation du SSR qui est utilisé pour gagner des performances	Dans le cadre du projet, le rendu serveur apporte peu de valeur
Typage	6/10	Typescript natif et évite les erreurs de type	Nécessite de l'apprentissage si peu d'expérience (React/Next)
File routing	5/10	Pas besoin de maintenir un fichier à part	L'arborescence devient plus conséquente
Complexité	6/10	Bien documenté en ligne et déjà utilisé par le passé	Pas vu durant la formation, ce qui nécessite un apprentissage supplémentaire
Utilisation	9/10	Utilisation lors de projets personnels et lors du P-APPRO2	-

1.5.4.2 Alternatives

Alternative	Poids	Stack	Pourquoi	Limite
ASP.NET	1/10	C#	Typage fort avec le C# + Framework bien documenté + Architecture MVC.	Hors contrainte : le cahier des charges impose JavaScript / Node.js
Vue JS + Express	5/10	JS	Courbe d'apprentissage moins grande car vue durant la formation	Deux projets distincts à gérer (CORS, déploiement, ports)
React + Express	5.5/10	JS/TS	Utilisation lors de projet personnel, Légèreté d'express	Deux projets distincts à gérer (CORS, déploiement, ports), routing plus important et pas de SSR natif : application entièrement client-side

1.5.4.3 Bibliothèques utilisées

Nom	Version	But
bcrypt	^6.0.0	Hash des données
dompurify	^3.4.2	Nettoyage du HTML afin d'éviter les failles XSS
marked	^18.0.2	Conversion du Markdown en HTML
dotenv	^17.4.2	Utilisation de variables d'environnement
jsonwebtoken	^9.0.3	Génération des tokens JWT
Mui/joy	^5.0.0-beta-52	Bibliothèque de composants React
prisma	^6.12.0	ORM permettant de contrer les injections SQL
React-icons	^5.6.0	Bibliothèque d'icônes

L'ensemble des bibliothèques utilisées dans le projet ont été sélectionnées dans leurs versions les plus récentes et stables disponibles lors de la création du projet.

Un audit de sécurité effectué via `npm audit` a révélé deux vulnérabilités de sévérité modérée liées à PostCSS (< 8.5.10), permettant une faille XSS lors de la génération de CSS à partir de contenu non fiable. Elles proviennent d'une dépendance imbriquée de Next.js (`node_modules/next/node_modules/postcss`), pour laquelle aucun correctif n'est disponible sans un retour en arrière majeur des versions de Next.js.

```
PS C:\Users\pk88yte\Documents\Github\Agora> npm audit
# npm audit report

postcss <8.5.10
Severity: moderate
PostCSS has XSS via Unescaped </style> in its CSS Stringify Output -
https://github.com/advisories/GHSA-qx2v-qp2m-jg93
fix available via `npm audit fix --force`
Will install next@9.3.3, which is a breaking change
node_modules/next/node_modules/postcss
  next 9.3.4-canary.0 - 16.3.0-canary.5
  Depends on vulnerable versions of postcss
  node_modules/next

2 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
npm audit fix --force
```

Figure 23. Résultat du npm audit

Cependant, cette vulnérabilité n'impacte pas le code de l'application, car aucun plugin supplémentaire n'est importé, à l'exception de celui utilisé par Tailwind CSS, inclus par défaut lors de la création du projet Next JS

1.5.5 Authentification

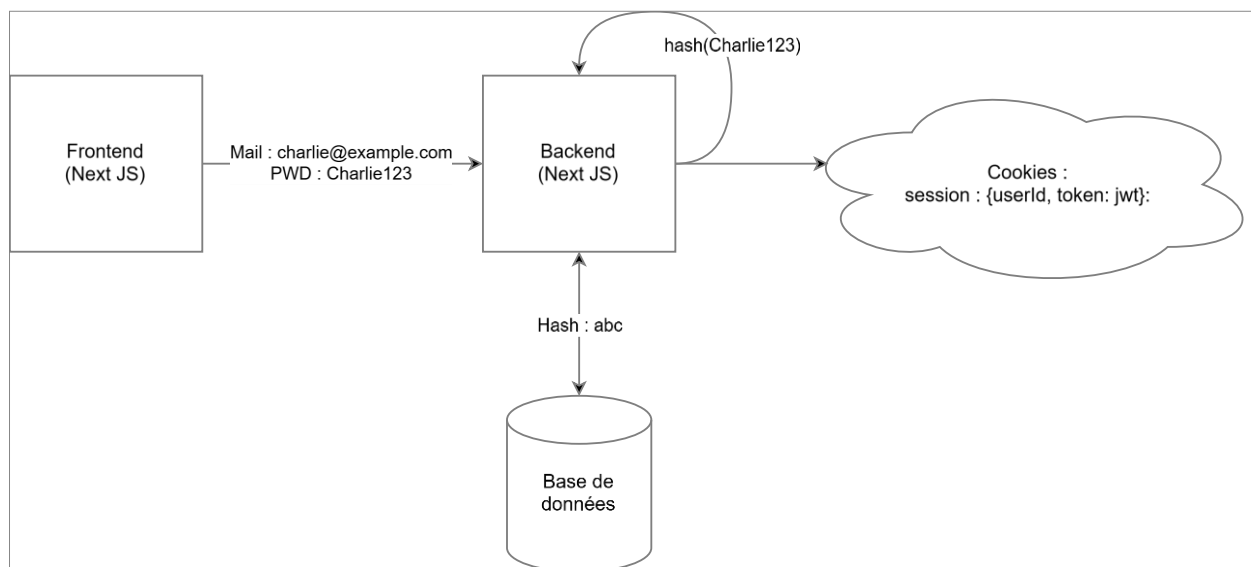


Figure 24. Schéma d'authentification d'un admin sur l'app

Ce diagramme illustre un système d'authentification classique en 4 étapes. En premier lieu, l'utilisateur rentre ses identifiants de connexion (e-mail : charlie@example.com, mot de passe : Charlie123). Ensuite, ces informations sont transmises au backend qui va faire trois choses :

1. Hasher le mot de passe
2. Récupérer le hash stocké dans la base de données pour cet utilisateur
3. Comparer les deux hashes

Si les deux hashes correspondent alors un cookie HTTP sécurisé (httpOnly : true, secure : true, sameSite : "strict") va être créé et va stocker l'identifiant de l'utilisateur ainsi que son token JWT. Cependant si les deux hash ne correspondent pas, une erreur est retournée.

1.5.6 Script (CRON)

Les schémas illustrent le fonctionnement du système d'envoi de newsletters automatisé.

Étape 1 :

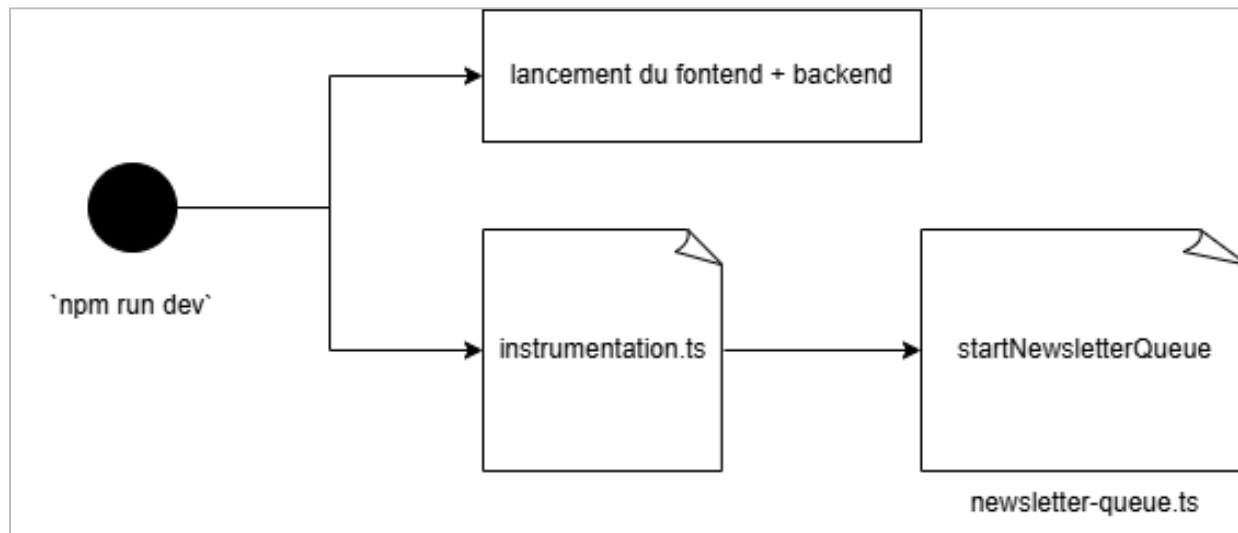


Figure 25. Schéma de l'exécution de l'application

Lors du démarrage de l'application, une phase d'initialisation lance à la fois le frontend, le backend ainsi qu'un processus d'instrumentation qui démarre une file de traitement (newsletter-queue).

Étape 2 :

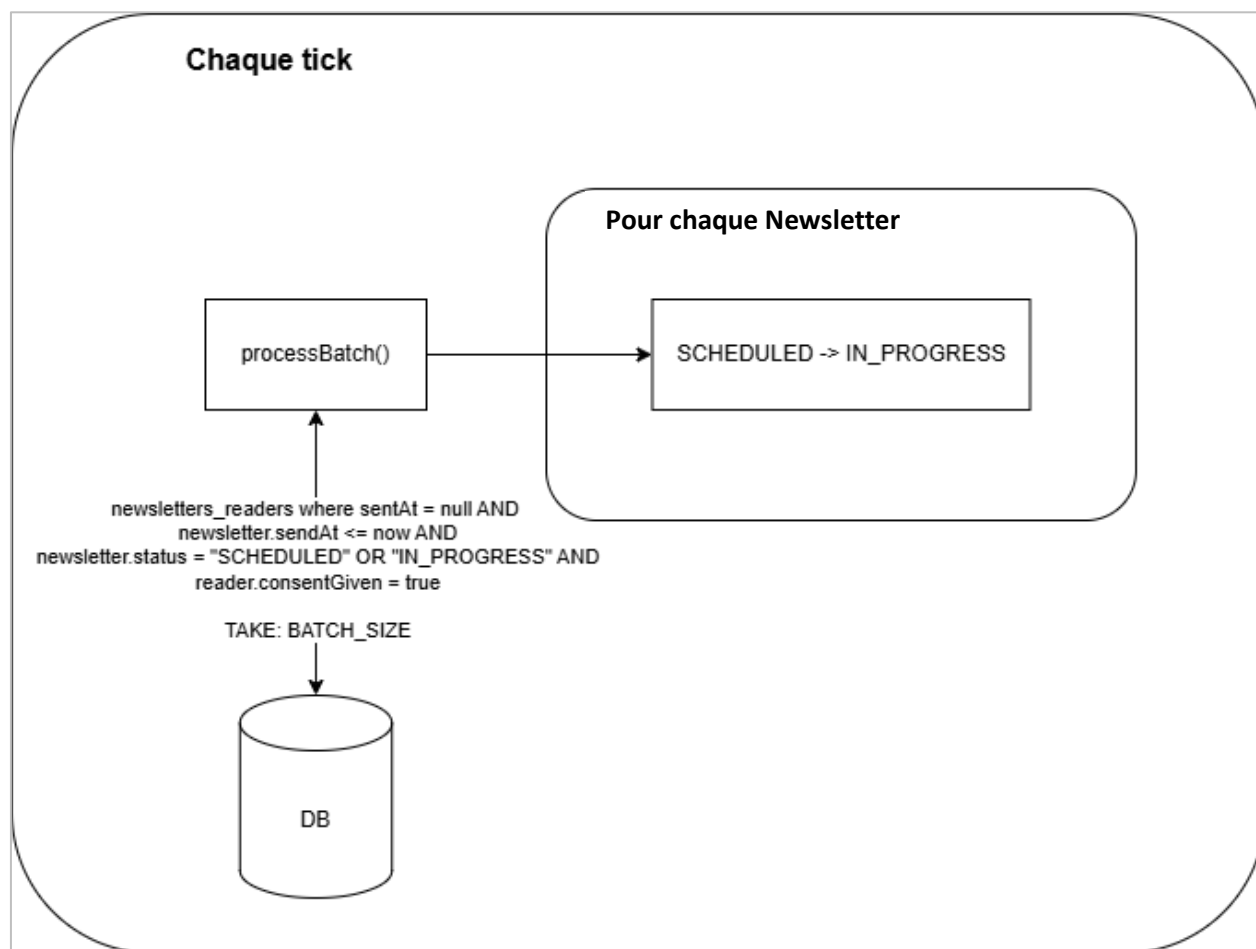


Figure 26. Schéma montrant le changement de statut d'une campagne

À intervalles réguliers (tick=60000 ms par défaut mais modifiable en modifiant le fichier .env), la fonction `processBatch()` est exécutée. Elle récupère en base de données un lot limité (`BATCH_SIZE`) de newsletters dont la date d'envoi est atteinte ($\text{sendAt} \leq \text{now}$), ayant un statut "**SCHEDULED**" ou "**IN_PROGRESS**", et dont les destinataires ont donné leur consentement. Pour chaque newsletter, le statut passe de "**SCHEDULED**" à "**IN_PROGRESS**".

Étape 3 :

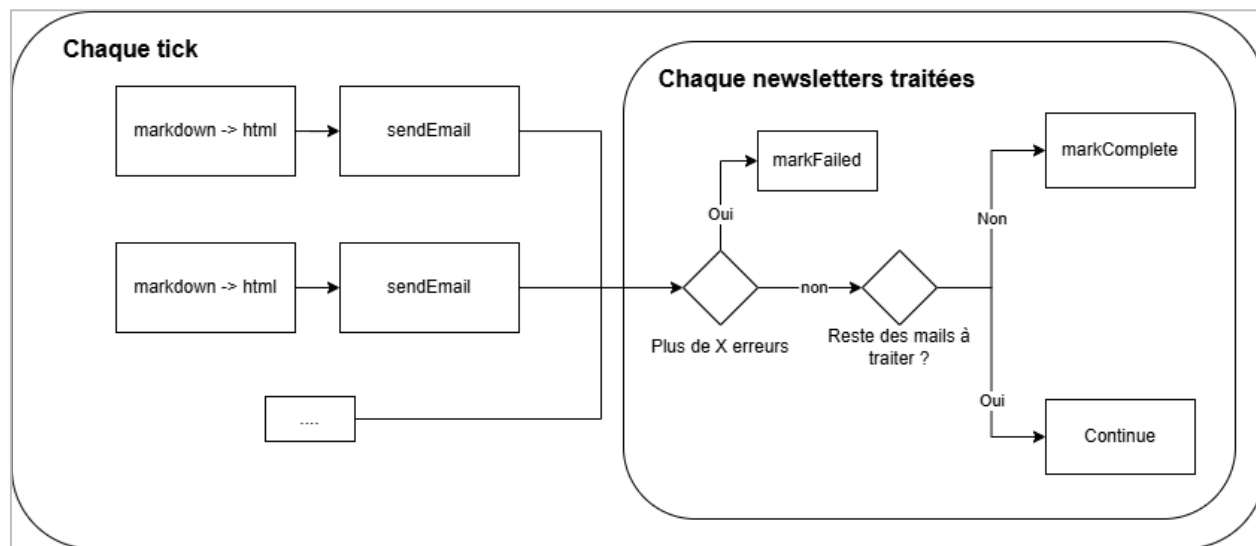


Figure 27. Schéma de l'envoi des mails

Les e-mails sont alors traités par lots : le contenu est converti de Markdown en HTML avant d'être envoyé. Les envois d'e-mails au sein d'un même lot sont effectués en parallèle, ce qui permet d'optimiser les performances et de réduire le temps total de traitement. Après, pour chaque newsletter qui a été traitée, l'application vérifie en premier lieu s'il y a eu des erreurs lors des envois et s'il y en a plus de X (valeur définie dans les variables d'environnement par défaut, c'est 5), alors la campagne est notée comme "**FAILED**". Dans un deuxième temps, l'application vérifie s'il reste des e-mails à envoyer. Si oui, le traitement continue. Sinon, la newsletter est marquée comme "**COMPLETE**" si tout s'est bien déroulé.

1.6 Planification et suivi

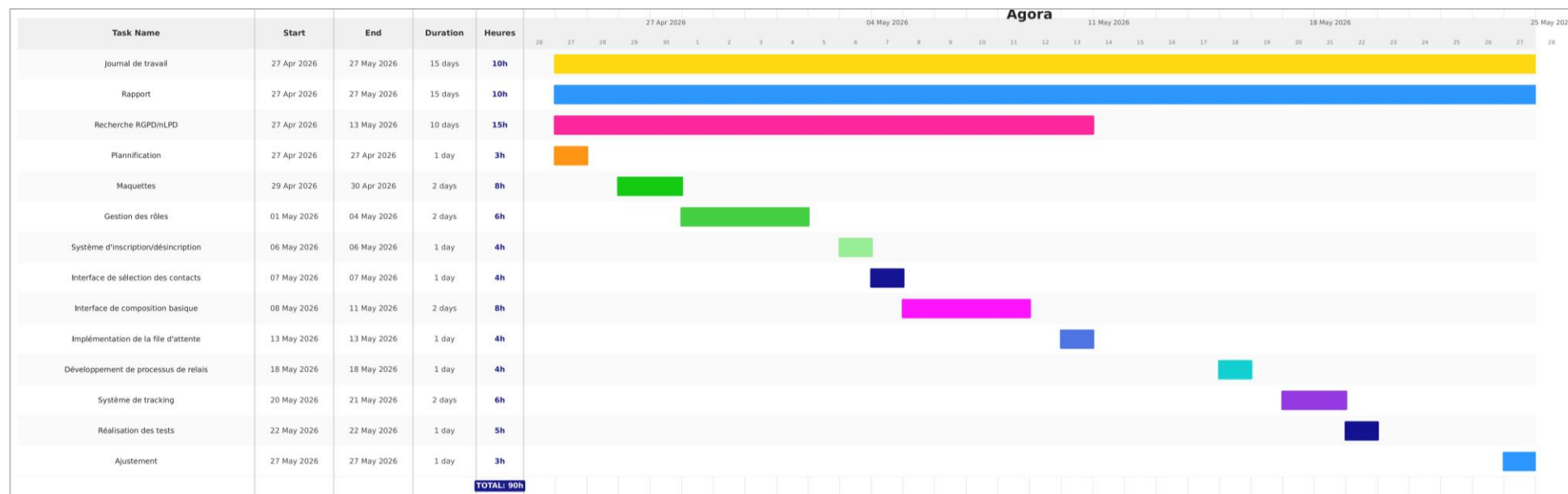


Figure 28. Planification initiale

Le « Journal de travail » est partagé dans les annexes

1.7 Réalisation

1.7.1 Structure du dépôt

Dans le cadre de ce projet de TPI, une stratégie de gestion du dépôt Git a été mise en place afin d'assurer un suivi efficace du développement et une bonne traçabilité des modifications.

Étant donné qu'il s'agit d'un projet individuel, l'utilisation d'une stratégie de branches complexe n'était pas nécessaire. Par conséquent, l'ensemble du développement a été réalisé directement sur la branche principale (main). Cette approche a permis de simplifier la gestion du projet tout en restant adaptée à sa taille et à son contexte.

L'utilisation de branches supplémentaires (comme *feature*, *develop* ou *hotfix*) aurait été excessive dans ce cas précis, car il n'y avait ni collaboration simultanée, ni besoin de gérer des intégrations multiples ou des conflits entre plusieurs développeurs.

Cependant, dans le cadre d'un projet en équipe, une stratégie plus structurée aurait été mise en place, telle que Git Flow.

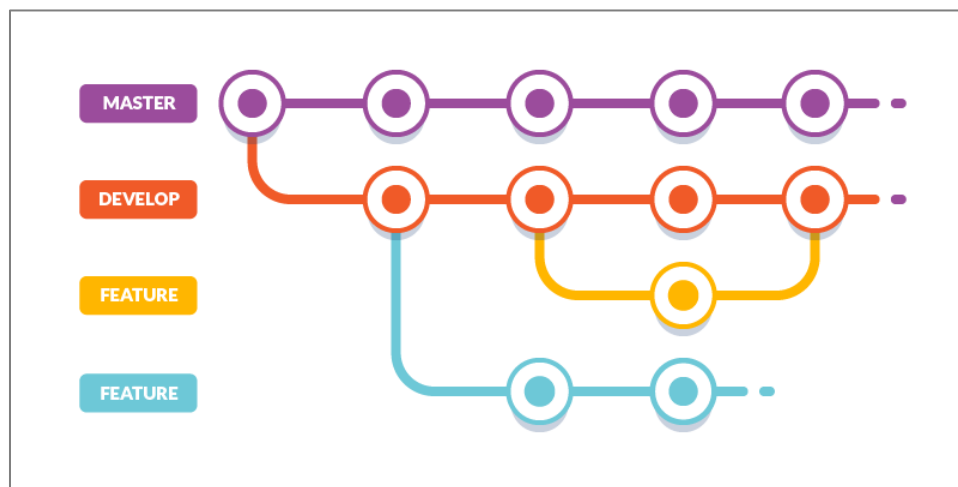


Figure 29. Schéma d'un git flow

Cette approche permet de :

1. Séparer clairement les environnements de développement (*develop*) et de production (main)
2. Travailler sur des fonctionnalités via des branches dédiées (*feature/**)
3. Gérer les corrections urgentes avec des branches *hotfix*
4. Organiser les phases de livraison avec des branches *release*

Une telle stratégie améliore la collaboration, la lisibilité du projet et la stabilité du code dans un contexte multi-développeurs.

1.7.2 Version des outils utilisés

Le développement du projet a été réalisé sur le poste de travail local.

Outil / Environnement	Version locale	Rôle
Système d'exploitation	Windows 10 Education 22H2	Environnement de développement local
Node.js	V24.13.0	Runtime JavaScript côté serveur
Npm	V11.6.2	Gestionnaire de paquets Node
Git	2.50.0.windows.1	Outils de versioning local
MariaDB	V11.4	Système de base de données

1.8 Tests et validation

1.8.1 Stratégie de tests

L'objectif de cette stratégie est de garantir la fiabilité, la stabilité et la conformité du système d'envoi de newsletters. Les tests portent principalement sur le comportement du service d'envoi d'e-mails, la gestion des consentements utilisateurs, le suivi des interactions et la gestion des erreurs techniques. Les tests mis en place sont des tests unitaires permettant de vérifier la résilience de l'application.

1.8.2 Cas de tests

Titre du test	Objectif
Envoi à lecteur inscrit	Vérifier qu'une newsletter est correctement envoyée à un lecteur avec consentement actif
Tracking des clics	Vérifier que les liens de tracking sont correctement injectés dans le contenu HTML
Envoi en batch	Vérifier l'envoi simultané à plusieurs lecteurs
Lien de désinscription	Vérifier la présence du lien de désinscription
Pas d'envoi sans consentement	Garantir qu'aucun e-mail n'est envoyé sans consentement utilisateur
Token test_token_invalide	Vérifier la gestion spécifique des erreurs HTTP 401
Vérification des statuts	Vérifier la mise à jour correcte des statuts d'envoi
Détection statut SCHEDULED	Vérifier le comportement lorsque aucune newsletter à traiter n'est trouvée

1.8.3 Tests manuels

1.8.3.1 Exécution

Tous les tests manuels ont été exécutés sur l'environnement de staging (<https://agora.w3.pm2etml.ch/>) afin de vérifier le bon comportement de l'application dans un environnement proche d'un environnement de production.

1.8.3.2 Jeux de tests

Titre	Contexte	Action	Résultat attendu	Statut
Envoi à lecteur inscrit	Campagne avec un statut planifié avec un lecteur sélectionné : - nogahe2626@nuitx.com	Attendre que la date d'envoi de la campagne soit dépassée et que les e-mails soient envoyés	E-mail reçu	✓
Tracking des clics	E-mail de la campagne "Actualités du mois d'avril" reçu	Cliquer sur un lien présent dans l'e-mail	Clique enregistré	✓
Envoi en batch	Plusieurs lecteurs inscrits sélectionnés : - nogahe2626@nuitx.com - thomas.nardou@eduvaud.ch	Attendre que la date d'envoi de la campagne soit dépassée et que les e-mails soient envoyés	E-mail reçu	✓
Pas d'envoi sans consentement	Lecteur sans consentement actif : - corima6175@nuitx.com	Attendre que la date d'envoi de la campagne soit dépassée et que les e-mails soient envoyés	E-mail non reçu	✓
Token invalide	Token pour l'API mail invalide : test_token_invalide	Attendre que la date d'envoi de la campagne soit dépassée	Campagne mise en statut "Échouée"	✓
Vérification des statuts	Campagne avec un statut planifié avec 5 lecteurs et un BATCH_SIZE de 2	Attendre que la date d'envoi de la campagne soit dépassée	Campagne mis en statut "En cours"	✓
Vérification des statuts	Campagne avec un statut planifié avec 5 lecteurs, un BATCH_SIZE de 2 et une campagne avec un statut "En cours"	Attendre que les e-mails soient partis	Campagne mis en statut "Terminée" ou "Échouée"	✓
Détection statut SCHEDULED	Aucune campagne à traiter (Date d'envoi > Date actuelle)	Attendre l'exécution du script d'envoi	Aucune campagne n'est traitée et aucun envoi est fait	✓

1.8.4 Tests unitaires

1.8.4.1 Exécution

Les tests unitaires sont exécutés localement via la commande :

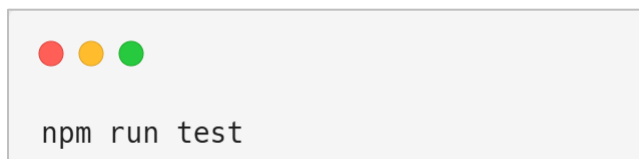


Figure 30. Commande pour exécuter les tests

Cette commande permet de lancer l'ensemble de la suite de tests et de vérifier le bon fonctionnement du module avant toute mise en production.

Dans une prochaine version, il pourrait être envisagé d'automatiser l'exécution des tests via une GitHub Action. Cela permettrait de lancer automatiquement les tests unitaires à chaque **"push"** et à chaque **"pull request"**, afin de détecter rapidement les régressions, sécuriser les livraisons et garantir la stabilité du projet avant toute fusion de code. L'exécution des tests continuerait également à être possible localement, permettant de vérifier le bon fonctionnement du module avant la mise en production.

1.8.4.2 Jeux de tests

Titre du test	Description	Résultat attendu	Statut
Envoi à lecteur inscrit	Envoi d'une newsletter à un lecteur avec consentement actif	E-mail envoyé via Maily, newsletter marquée COMPLETED, findMany/findFirst/update/count appelés	✓
Tracking des clics	Vérification que les liens de tracking sont inclus dans le HTML de l'e-mail	HTML contient "api/newsletters/track/clicked", tous les mocks appelés correctement	✓
Envoi en batch	Envoi à 5 lecteurs dans une même opération batch	5 e-mails envoyés avec les bonnes adresses, update appelé 5 fois	✓
Lien de désinscription	Vérification que le lien de désinscription est présent dans l'e-mail	HTML contient "unsubscribe?token=" avec le token, mocks vérifiés explicitement	✓
Pas d'envoi sans consentement	Les lecteurs sans consentement ne reçoivent pas d'e-mail	fetch n'est jamais appelé, findMany vérifié avec filtre consentGiven: true	✓
Token test_token_invalide	Test avec le token invalide spécifique "test_token_invalide"	sendEmail lance une erreur "Maily HTTP 401"	✓
Vérification des statuts	Vérification du traitement correct des statuts de newsletter	E-mail envoyé avec succès, update marquant sentAt vérifié avec clé composite	✓
Détection statut SCHEDULED	Vérification que le système détecte et traite les newsletters avec statut SCHEDULED	findMany retourne liste vide, aucun appel supplémentaire (early return)	✓

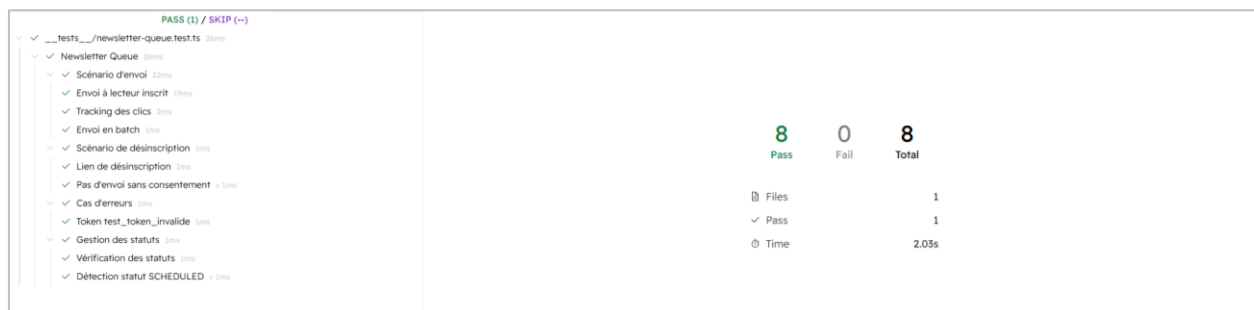
Preuve :

Figure 31. Preuve de la bonne exécution des tests

1.8.5 Bugs connus

Il existe deux scénarios différents pour ce bug :

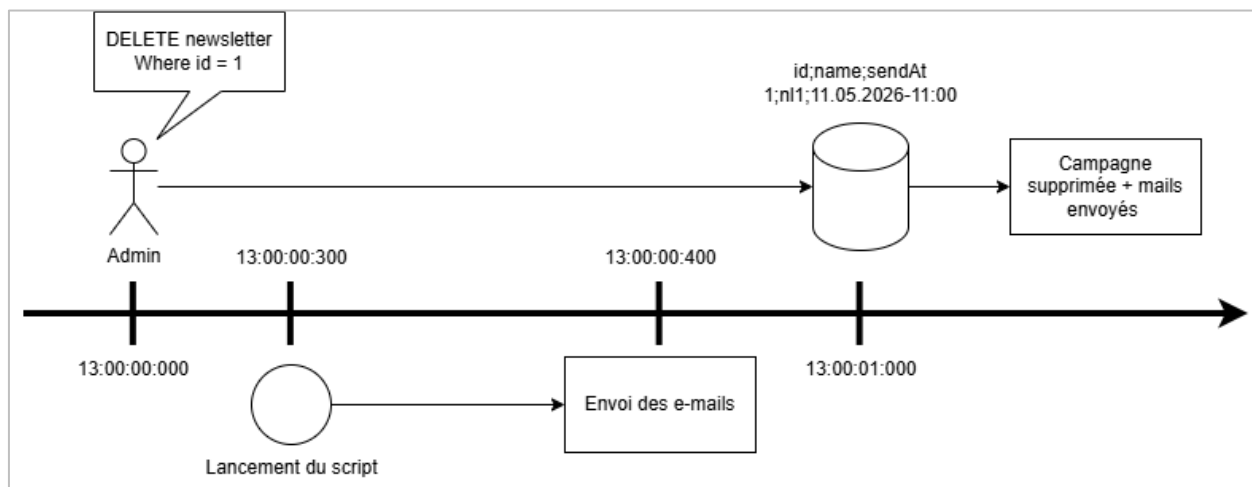
Scénario 1 :

Figure 32. Schéma du premier scénario

Le premier scénario surgit quand l'administrateur veut supprimer une campagne qui est planifiée (13:00:00:000) mais le script se lance (13:00:00:300) et envoie les mails aux lecteurs avant que la requête n'arrive à la base de données. Dans ce cas, l'administrateur se retrouve dans une situation où la campagne qui est censée être supprimée a été envoyée.

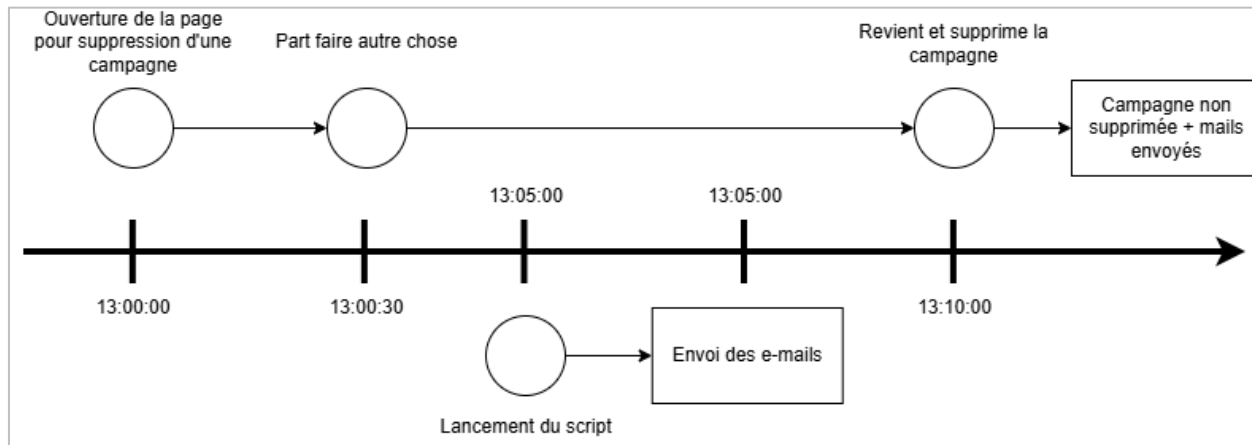
Scénario 2 :

Figure 33. Schéma du deuxième scénario

Dans ce second scénario, bien que similaire au premier sur le fond, il diffère dans sa forme. L'administrateur accède à la page d'accueil avec l'intention de supprimer une campagne. Cependant, il s'absente momentanément avant d'effectuer l'action. Pendant ce laps de temps, le script d'envoi s'exécute automatiquement et les e-mails associés à la campagne sont envoyés. Lorsque l'administrateur revient et tente finalement de supprimer la campagne, une erreur se produit, car la campagne a entre-temps changé d'état et possède désormais le statut "envoyée". Par conséquent, la campagne n'est pas supprimée alors que les e-mails ont déjà été transmis aux lecteurs.

Classification du bug :

- **Sévérité** : Mineur
- **Type** : Fonctionnel
- **Priorité** : Moyenne/Faible

1.9 Livraison

Selon le cahier des charges, il a été demandé de livrer à la fin du projet une planification initiale (remise le premier jour de travail), un rapport régulier aux experts tous les mercredis et vendredis par courriel contenant le rapport et le journal de travail, un rapport de projet structuré et une vidéo de démonstration.

Si une livraison plus importante devait avoir lieu, l'application aurait pu être livrée sous forme d'un Dockerfile avec une image publiée sur Docker Hub ou faire un déploiement automatique via les GitHub Actions.

1.10 Analyse critique

1.10.1 Application

L'application, dans son état actuel, est entièrement fonctionnelle. Toutefois, certains aspects pourraient encore être améliorés, notamment le système de suppression des campagnes. Cependant l'application pourrait différencier la suppression selon le statut de la campagne au moment de l'action. Par exemple, une campagne ayant le statut de brouillon pourrait être supprimée définitivement, comme c'est le cas actuellement. En revanche, une campagne planifiée pourrait simplement recevoir le statut "annulée" au lieu d'être supprimée. Cette approche permettrait de conserver certaines données pour les statistiques, tout en offrant la possibilité d'annuler cette action et de réactiver la campagne si nécessaire.

1.10.2 API mail vs SMTP

L'application actuelle utilise une API d'envoi d'e-mail et ce choix offre plusieurs avantages, notamment un contrôle sur les envois et la possibilité de mettre en place des mécanismes personnalisés comme une limite du débit d'envoi, la détection de potentiels spams, etc. Cette approche permet également une meilleure traçabilité des opérations grâce à des logs détaillés, facilitant ainsi le suivi des erreurs et l'audit des envois. En outre, l'intégration via une API simplifie l'architecture globale et s'adapte mieux aux environnements modernes orientés services, tout en offrant une flexibilité importante pour faire évoluer la logique d'envoi sans dépendre directement d'un serveur SMTP.

En comparaison, l'utilisation de SMTP repose sur un protocole standardisé et largement compatible, ce qui le rend particulièrement robuste dans des systèmes variés et interconnectés. Il a l'avantage de déléguer la gestion des envois au serveur SMTP, notamment les nouvelles tentatives en cas d'échec, ce qui réduit la charge côté application. SMTP est également adapté aux volumes importants d'e-mails grâce à sa bonne capacité de scalabilité, et il fournit des mécanismes essentiels de retour comme les bounce messages permettant de suivre efficacement la délivrabilité des messages.

1.10.3 Configuration DNS

L'analyse de la configuration DNS s'est principalement concentrée sur la zone DNS ainsi que sur les mécanismes de sécurité SPF et DMARC. Cette étude porte sur le nom de domaine section-inf.ch, utilisé par l'API d'envoi d'e-mails.

Serveurs DNS autoritaires :

Nom de domaine	Adresses IP
ns81.kreativmedia.ch.	80.74.155.167
	2a00:1128:0:157::15
ns82.kreativmedia.ch.	80.74.134.150
	2a00:1128:0:202:1::22

L'analyse des enregistrements NS montre que les serveurs "ns81.kreativmedia.ch" et "ns82.kreativmedia.ch" sont bien ceux qui gèrent la configuration DNS du domaine "section-inf.ch".

36	2a00:1128:0:157::15	UDP	section-inf.ch.	NS	NOERROR
37	80.74.134.150	UDP	section-inf.ch.	NS	NOERROR
38	2a00:1128:0:202:1::22	UDP	section-inf.ch.	NS	NOERROR
39	80.74.155.167	UDP	section-inf.ch.	NS	NOERROR

Figure 34. Enregistrements DNS NS

Configuration SPF et DMARC :

Le domaine dispose d'un enregistrement SPF configuré au niveau du serveur DNS. Cette politique permet de restreindre l'envoi d'e-mails aux seuls serveurs de messagerie autorisés appartenant au domaine kreativmedia.ch. Cette configuration contribue à limiter les risques d'usurpation d'identité et de spoofing des e-mails.

SPF Policy Information Main policy	
Location	section-inf.ch
v	spf1
include	_spf.kreativmedia.ch
mx	
a	
-all	

Figure 35. Informations sur la politique SPF

Le domaine dispose également d'un enregistrement DMARC permettant de renforcer la sécurité des échanges de courrier électronique. Cette politique complète le SPF en définissant le comportement à adopter lorsqu'un mail échoue aux vérifications d'authentification.

DMARC Policy Information	
Policy location	_dmarc.section-inf.ch
v	DMARC1
p	quarantine

Figure 36. Informations sur la politique DMARC

De ce cas, si un e-mail n'est pas reconnu par la politique SPF, alors celui-ci sera mis en "quarantaine", ce qui veut tout simplement dire que le mail ne sera pas envoyé.

Par exemple, en analysant les headers de deux mails envoyés : un avec une adresse personnelle et l'autre avec une adresse ...@section-inf.ch.

Comme indiqué sur l'image, le premier e-mail a été soumis par un système ayant comme nom de domaine "outbound.mr.icloud.com" donc cet e-mail sera mis en quarantaine et ne sera pas envoyé car il ne sera pas reconnu par la politique SPF.

Hop	Submitting host	Receiving host
1	smtpclient.apple (unknown [17.57.152.38])	p00-icloudmta-asmtplib-us-west-2a-100-percent-4 (Postfix)
2	outbound.mr.icloud.com (unknown [127.0.0.2])	p00-icloudmta-asmtplib-us-west-2a-100-percent-4 (Postfix)
3	outbound.mr.icloud.com (57.103.70.53)	GV1PEPF000006FB.mail.protection.outlook.com (10.167.240.7)
4	GV1PEPF000006FB.CHEP278.PROD.OUTLOOK.COM (2603:10a6:710:2b::78)	GV0P278CA0074.outlook.office365.com (2603:10a6:710:2b::7)
5	GV0P278CA0074.CHEP278.PROD.OUTLOOK.COM (2603:10a6:710:2b::7)	GVAP278MB0932.CHEP278.PROD.OUTLOOK.COM (2603:10a6:710:46::12)
6	GVAP278MB0932.CHEP278.PROD.OUTLOOK.COM (2603:10a6:710:46::12)	ZR5P278MB1761.CHEP278.PROD.OUTLOOK.COM

Figure 37. Header de l'e-mail envoyé depuis une adresse personnel (@icloud.com)

Contrairement à la première image, cet e-mail a été soumis par un système avec un nom de domaine valide. Celui-ci va donc bien passer la vérification SPF et être envoyé.

Hop.	Submitting host	Receiving host
1		elara.kreativmedia.ch (Postfix, from userid 11056)
2	elara.kreativmedia.ch (80.74.155.167)	ZR2PEPF0000012E.mail.protection.outlook.com (10.167.241.38)
3	ZR2PEPF0000012E.CHEP278.PROD.OUTLOOK.COM (2603:10a6:910:52:cafe::a9)	ZR2P278CA0070.outlook.office365.com (2603:10a6:910:52::13)
4	ZR2P278CA0070.CHEP278.PROD.OUTLOOK.COM (2603:10a6:910:52::13)	GVAP278MB0860.CHEP278.PROD.OUTLOOK.COM (2603:10a6:710:47::14)
5	GVAP278MB0860.CHEP278.PROD.OUTLOOK.COM (2603:10a6:710:47::14)	ZR5P278MB1761.CHEP278.PROD.OUTLOOK.COM

Figure 38. Header de l'e-mail envoyé depuis une adresse valide (@section-inf.ch)

1.11 Conclusion et bilan personnel

Ce projet avait pour objectif de développer une application de plateforme de diffusion de newsletters. Les deux principaux objectifs étaient, d'une part, la mise en place d'un système d'envoi automatique et asynchrone des mails aux lecteurs et, d'autre part, la conformité de l'application aux réglementations RGPD et LPD. Ces deux objectifs ont pu être atteints avec succès.

Bien que l'application réponde aujourd'hui aux besoins essentiels définis au début du projet, elle présente des points forts mais aussi plusieurs axes d'amélioration. Parmi les points positifs, l'application propose une interface moderne et intuitive, facilitant sa prise en main et son utilisation. Elle répond aux exigences définies dans le cahier des charges, notamment grâce à la mise en place de l'envoi automatique et asynchrone des newsletters, ainsi qu'au respect des contraintes liées au RGPD et à la LPD.

Cependant, certains points sont à améliorer. Tout d'abord, aucun protocole HTTPS n'a encore été mis en place, ce qui représente un aspect important à considérer dans le cadre d'un futur déploiement en production. De plus, l'application ne dispose pas encore d'une page de statistiques avec des graphiques permettant de visualiser les performances des campagnes ou l'activité des utilisateurs. Il n'existe également aucun moyen de connaître précisément la prochaine exécution du script d'envoi, ce qui limite la visibilité sur l'automatisation des tâches. Un autre point serait la modification du système de suppression, comme indiqué un peu plus tôt.

Ce projet a présenté plusieurs aspects positifs. Tout d'abord, il m'a permis de travailler sur une application qui doit se reposer sur une base légale, ce qui m'a permis d'en apprendre plus sur ce domaine, ce qui est très formateur et se rapproche davantage du monde professionnel car la réalisation de projet devra toujours se baser sur une base légale. De plus, ce projet m'a donné l'occasion d'approfondir mes connaissances avec Next JS qui est un framework très utilisé aujourd'hui dans le monde professionnel.

Lors de la réalisation du projet, j'ai rencontré plusieurs problèmes, notamment au début. En effet, lors de la création du projet, j'avais installé des versions instables de Next.js et de Prisma, ce qui a entraîné de nombreuses erreurs, comme des problèmes de syntaxe ou encore l'impossibilité d'exécuter le seed de la base de données. Afin de pouvoir avancer efficacement dans le développement, j'ai finalement décidé d'utiliser les versions que j'avais déjà employées lors de mon projet P-APPRO2, qui étaient quant à elles plus stables.

Pour terminer, plusieurs évolutions pourraient être envisagées, notamment un tracking plus précis (IP, lecteur, etc). Cependant, rajouter cela demanderait un consentement supplémentaire de la part du lecteur. Un autre ajout serait la page des statistiques avec le nouveau système de suppression.

2 Partie 2 - Documentation du produit

2.1 Prérequis

1. Node.js
2. Npm ou yarn
3. Une base de données MariaDB ou MySQL (docker-compose disponible)
4. Git, Github desktop ou autre outil compatible avec Git

2.2 Installation / Déploiement

2.2.1 Clonage du repository



```
git clone https://github.com/ThomNardou/Agora.git  
cd Agora
```

Figure 39. Commande pour cloner le repository

2.2.2 Installation des dépendances

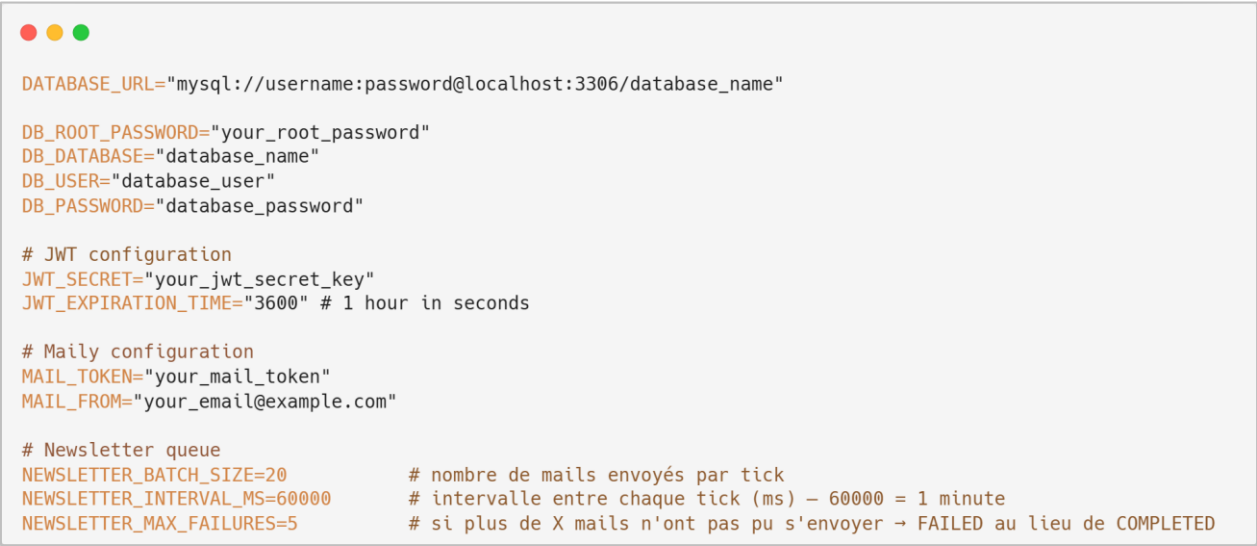


```
npm install
```

Figure 40. Commande pour installer les dépendances

2.2.3 Configurer les variables d'environnement

Créer un fichier `.env` sur la base du `.env.example` à la racine du projet et remplissez les variables.



```
DATABASE_URL="mysql://username:password@localhost:3306/database_name"

DB_ROOT_PASSWORD="your_root_password"
DB_DATABASE="database_name"
DB_USER="database_user"
DB_PASSWORD="database_password"

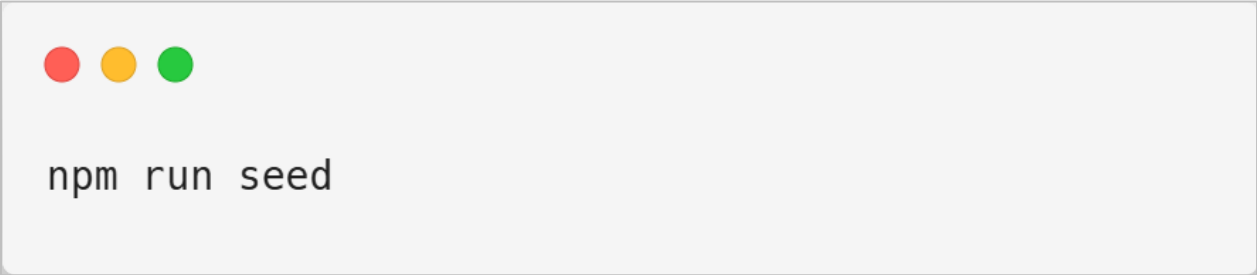
# JWT configuration
JWT_SECRET="your_jwt_secret_key"
JWT_EXPIRATION_TIME="3600" # 1 hour in seconds

# Maily configuration
MAIL_TOKEN="your_mail_token"
MAIL_FROM="your_email@example.com"

# Newsletter queue
NEWSLETTER_BATCH_SIZE=20           # nombre de mails envoyés par tick
NEWSLETTER_INTERVAL_MS=60000      # intervalle entre chaque tick (ms) - 60000 = 1 minute
NEWSLETTER_MAX_FAILURES=5         # si plus de X mails n'ont pas pu s'envoyer → FAILED au lieu de COMPLETED
```

Figure 41. Fichier `.env.example`

2.2.4 Initialiser la base de données



```
npm run seed
```

Figure 42. Commande pour peupler la base de données

2.2.5 Démarrer l'application

HTTP :

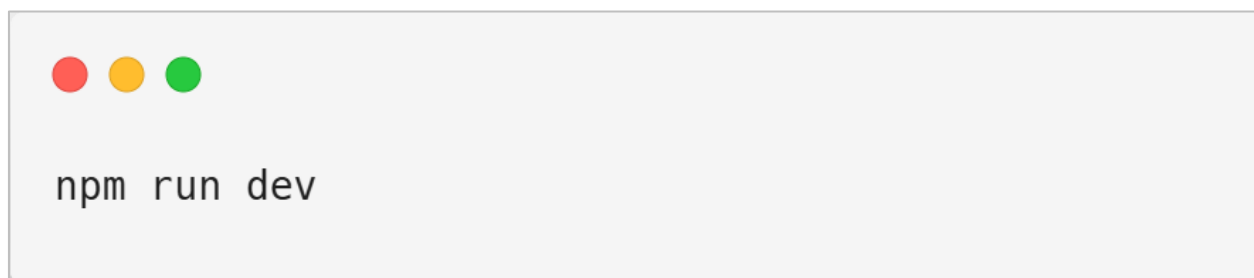


Figure 43. Commande pour démarrer l'application en http

HTTPS :

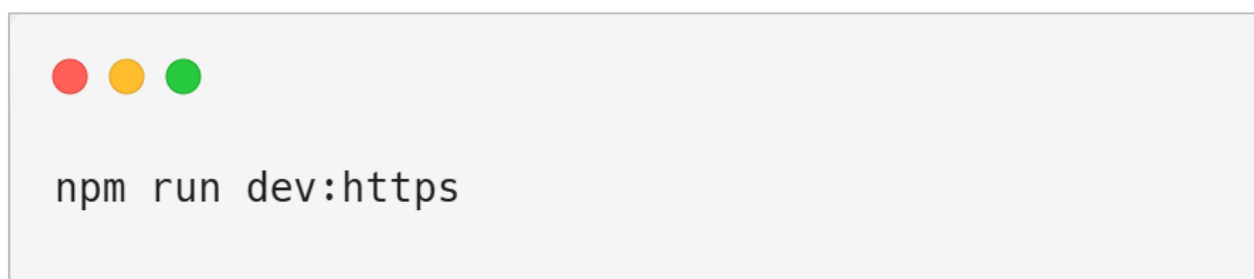


Figure 44. Commande pour démarrer l'application en https

2.2.6 Commandes disponibles

Commande	Description
npm run dev	Lance le serveur de développement
npm run dev:https	Lance le serveur avec HTTPS expérimental
npm run dev:seed	Réinitialise la DB et lance le serveur
npm run build	Construit l'application pour la production
npm run start	Lance l'application en production
npm run lint	Exécute ESLint
npm test	Exécute les tests unitaires (Vitest)
npm run test:ui	Lance l'interface de Vitest
npm run seed	Réinitialise et remplit la base de données
npm run db-push	Pousse le schéma Prisma à la base
npm run db-pull	Récupère le schéma actuel de la base
npm run db-generate	Génère le client Prisma

2.3 Guide d'utilisation

2.3.1 Inscription



The screenshot shows a subscription form for 'Agora'. At the top is a blue envelope icon, followed by the text 'Agora' and 'S'abonner'. Below this is an 'Email' label and a text input field containing 'demo@example.com'. Under the input field is a checked checkbox with the text 'J'accepte de recevoir des newsletters de Agora par e-mail'. A blue 'S'abonner' button is positioned below the checkbox. At the bottom, in small text, it states: 'En vous abonnent, vous reconnaissez avoir pris connaissance de la [politique de confidentialité](#) d'Agora et acceptez le traitement de vos données personnelles conformément à celle-ci.'

Figure 45. Page d'inscription

Afin de s'inscrire il est nécessaire de rentrer son adresse e-mail et d'accepter de recevoir des newsletters.

2.3.2 Désinscription

Chaque newsletter contient un lien de désinscription unique.



Figure 46. Pied de page de chaque mail envoyé

En cliquant sur ce lien, l'utilisateur accède à une page qui permet de se désabonner immédiatement.

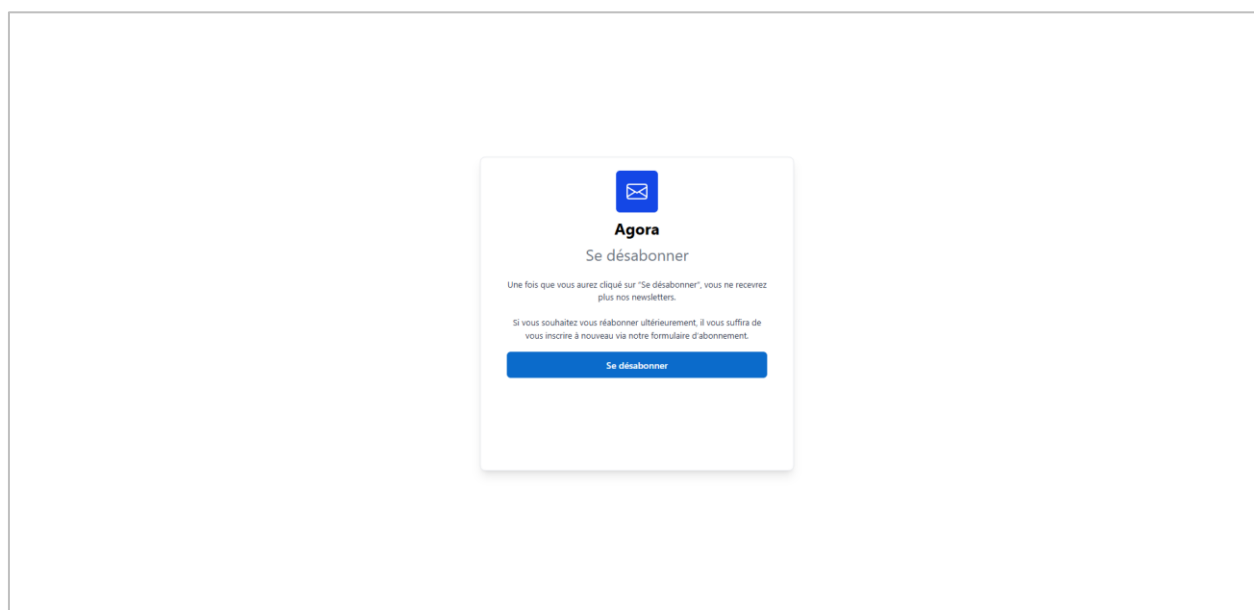


Figure 47. Page de désinscription

2.3.3 Connexion

Afin de se connecter au dashboard il est nécessaire de se rendre sur la page **"/login"**

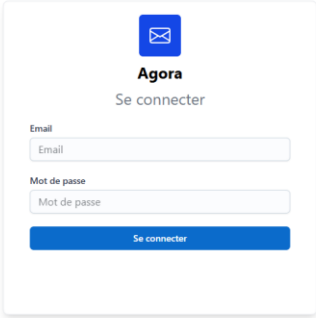
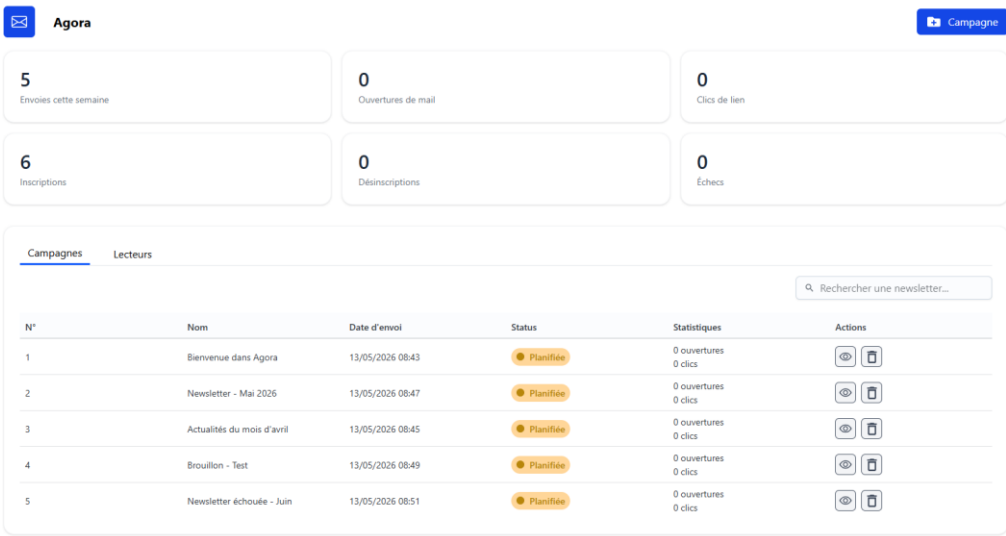


Figure 48. Page de login

Il est essentiel de saisir ses identifiants d'administrateur, puis d'appuyer sur le bouton se connecter. Une fois connecté, vous accédez au tableau de bord.



N°	Nom	Date d'envoi	Status	Statistiques	Actions
1	Bienvenue dans Agora	13/05/2026 08:43	Planifiée	0 ouvertures 0 clics	 
2	Newsletter - Mai 2026	13/05/2026 08:47	Planifiée	0 ouvertures 0 clics	 
3	Actualités du mois d'avril	13/05/2026 08:45	Planifiée	0 ouvertures 0 clics	 
4	Brouillon - Test	13/05/2026 08:49	Planifiée	0 ouvertures 0 clics	 
5	Newsletter échouée - Juin	13/05/2026 08:51	Planifiée	0 ouvertures 0 clics	 

Figure 49. Tableau de bord

2.3.4 Création d'une campagne

Pour créer une campagne, il est essentiel de se rendre d'abord sur la page de création de campagne. Pour y parvenir, il suffit d'appuyer sur le bouton "Campagne" dans le tableau de bord.

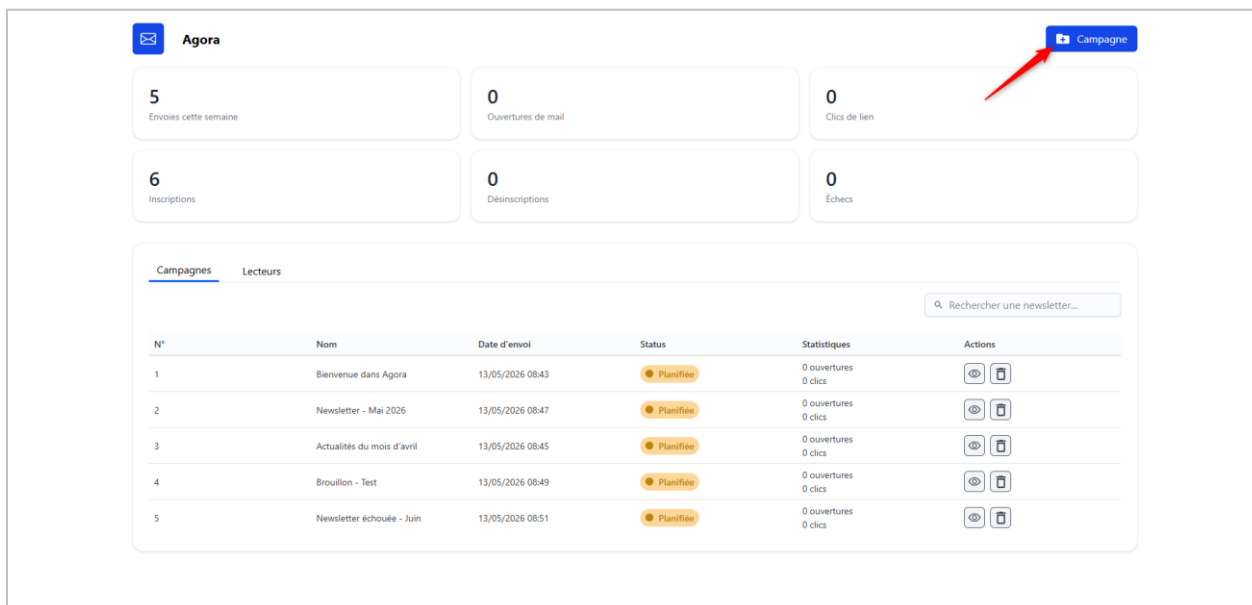


Figure 50. Tableau de bord

Une fois sur la page de création, il suffit de saisir le nom souhaité ainsi que le contenu du message. Pour modifier le style du texte, il est nécessaire de sélectionner la partie concernée puis cliquer sur le bouton correspondant au style désiré.

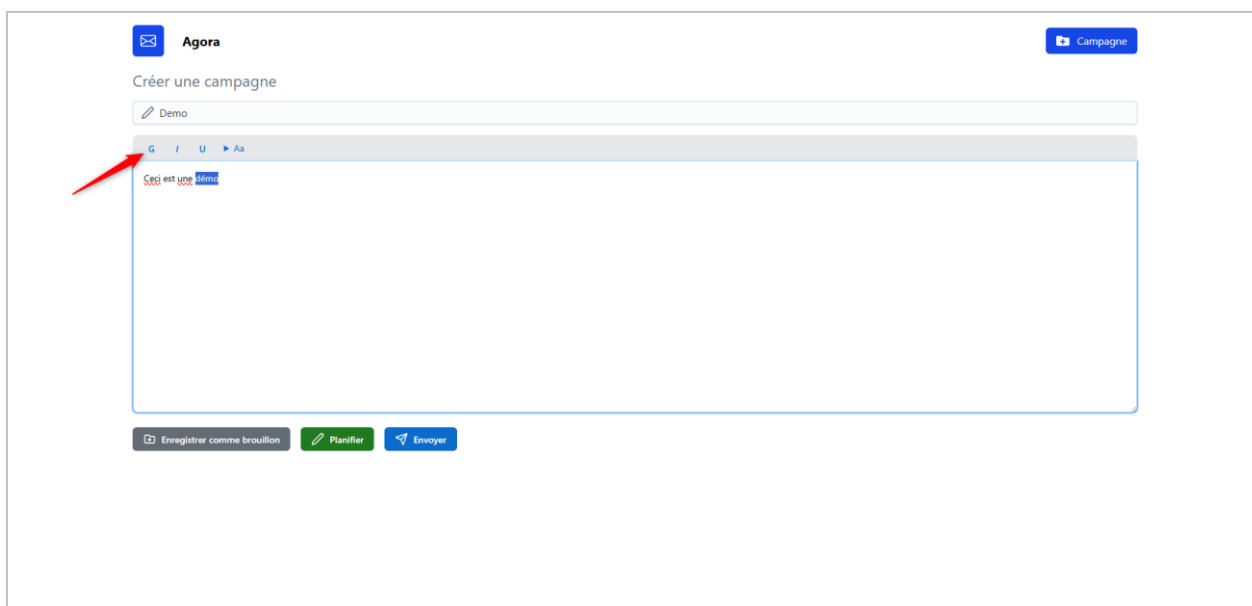


Figure 51. Page de création d'une campagne

2.3.4.1 Enregistrer comme brouillon

Une campagne peut être enregistrée comme brouillon. Cela veut dire que la campagne ne va être envoyée et que l'utilisateur peut modifier à tout moment le contenu de la campagne ou la supprimer.

Uniquement le nom est nécessaire pour pouvoir enregistrer.

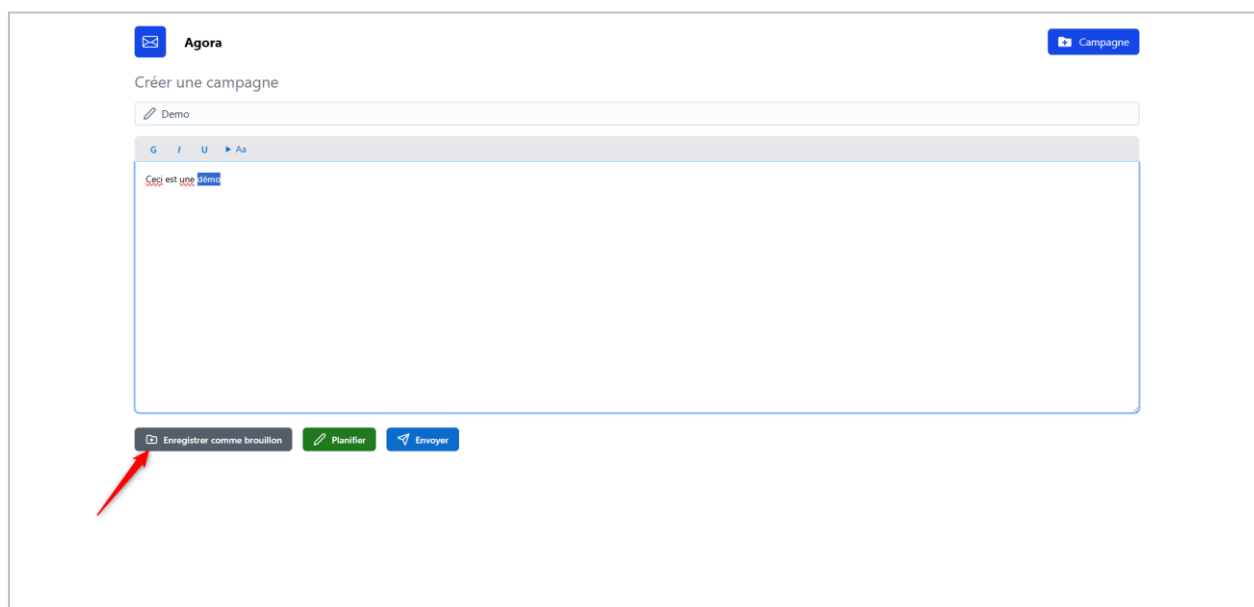


Figure 52. Page de création d'une campagne

2.3.4.2 Planifier l'envoi

Une campagne peut être aussi planifiée afin qu'elle soit envoyée plus tard. Pour cela, l'utilisateur doit appuyer sur le bouton "**planifier**". Ensuite, un popup va apparaître afin de rentrer la date et l'heure d'envoi de la campagne.

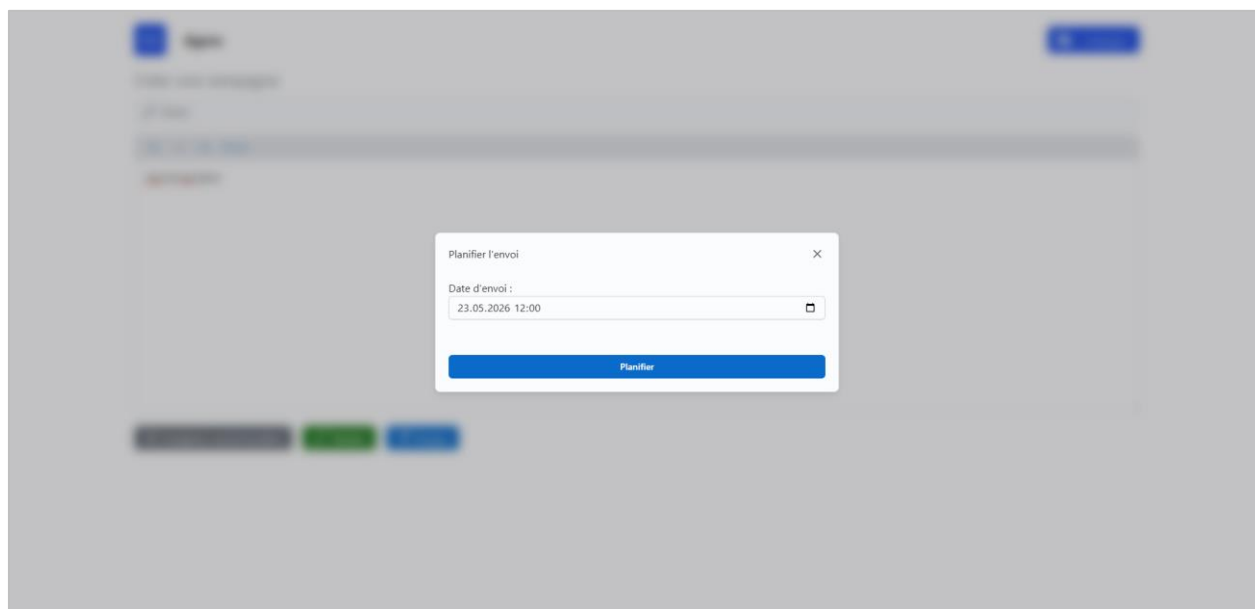


Figure 53. Popup de planification

Une fois la campagne planifiée, l'utilisateur doit choisir les lecteurs à qui la campagne va être envoyée.

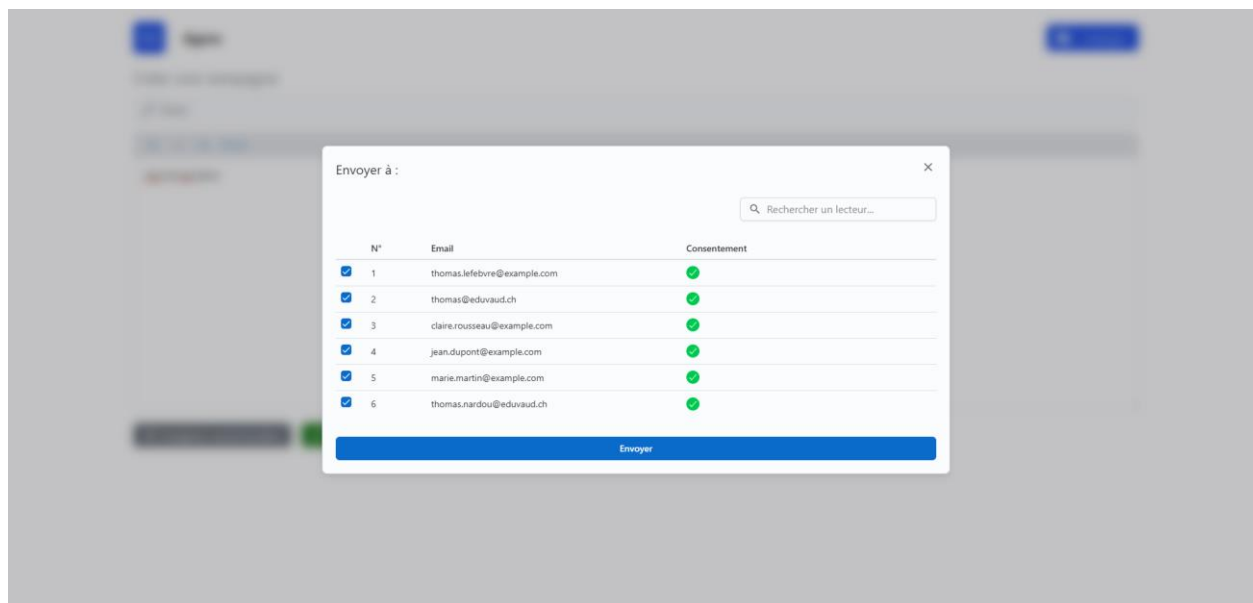


Figure 54. Pop-up de sélection des lecteurs

Une fois les lecteurs sélectionnés et après avoir appuyé sur le bouton "**Envoyer**", la campagne est alors enregistrée.

7	Demo	23/05/2026 12:00	● Planifiée	0 ouvertures 0 clics	 
---	------	------------------	-------------	-------------------------	---

Figure 55. Campagne planifiée

Contrairement au statut de brouillon, celui-ci ne permet pas de modifier la campagne, il peut être uniquement consulté ou supprimé.

2.3.4.3 Envoyer la campagne

Pour pouvoir envoyer la campagne il suffit d'appuyer sur le bouton "**Envoyer**". Ensuite, le popup de sélection de lecteur va s'afficher.

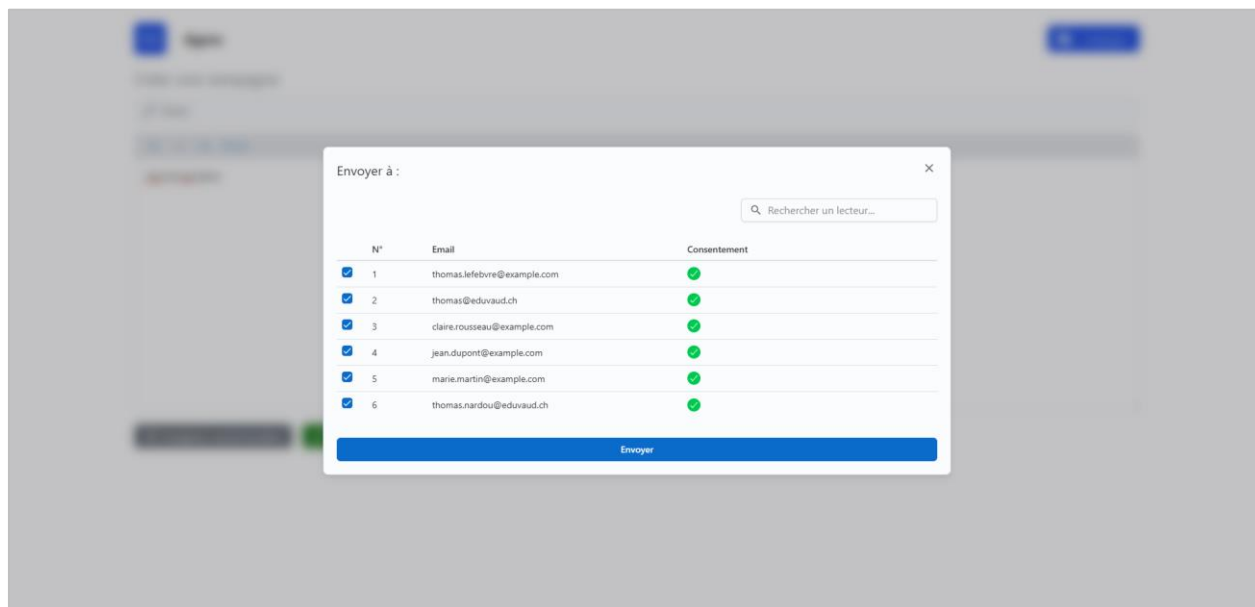


Figure 56. Popup de sélection des lecteurs

Une fois les lecteurs sélectionnés, l'utilisateur doit cliquer sur le bouton "**Envoyer**". La campagne est alors enregistrée avec le statut "Planifiée" et programmée pour être envoyée cinq minutes après l'enregistrement, afin de permettre à l'administrateur de la supprimer en cas d'erreur.

3 Résumé

3.1 Contexte et mandat

Agora est une application web de gestion de newsletters destinée aux administrateurs qui a été développée dans le cadre d'un Travail Pratique Individuel (TPI). Elle permet de créer, modifier et planifier des campagnes d'e-mails rédigées en Markdown, puis envoyées automatiquement aux abonnés. Basée sur JavaScript, Node.js et MariaDB, elle utilise une API d'envoi existante avec la fonction PHP "mail()" et un système de file d'attente pour assurer des envois fiables et asynchrones. L'application intègre une authentification sécurisée, une gestion des rôles, un formulaire d'inscription conforme à la LPD ainsi qu'un système de désinscription sécurisé. Elle propose également un suivi des performances grâce au tracking des ouvertures et des clics, tout en mettant l'accent sur la sécurité et la protection des données.

3.2 Solution réalisée

L'application Agora a été développée en TypeScript avec le framework Next.js avec un rendu côté serveur, en s'appuyant sur une base de données MariaDB gérée via l'ORM Prisma. L'authentification des administrateurs repose sur des tokens JWT, et la gestion des rôles applique le principe du moindre privilège. La création de campagnes s'effectue via un éditeur Markdown, avec un cycle de vie complet : brouillon, planifiée, en cours, terminée ou échouée. L'envoi des newsletters est asynchrone, piloté par un script qui traite les campagnes planifiées par lots via l'API d'envoi existante. Un système de tracking mesure les ouvertures par pixel invisible et les clics par réécriture dynamique des liens. La gestion des abonnements est conforme au RGPD/LPD, avec un consentement explicite à l'inscription et un mécanisme de désinscription sécurisé par tokens uniques.

3.3 Résultats principaux

Les résultats qui ont été obtenus montrent que l'application est entièrement fonctionnelle, intégrant toutes les fonctionnalités demandées. Le système d'authentification a été mis en place de manière sécurisée. L'application permet aussi un suivi de l'engagement des utilisateurs tout en restant conforme aux exigences du RGPD. Par la suite, une page avec un suivi plus détaillé pourrait être mise en place. Pour terminer, une interface utilisateur moderne et intuitive a été mise en place afin d'améliorer l'expérience utilisateur. Lors de la réalisation du projet, il a été mis en évidence qu'un conflit entre la suppression et l'envoi d'une campagne pourrait avoir lieu.

3.4 Conclusion

Pour conclure, l'application est une application de gestion de newsletters reposant sur une stack TypeScript et utilisant le framework next.js avec une base de données MariaDB. Toutes les fonctionnalités principales ont été réalisées avec un potentiel problème de conflit entre la suppression et l'envoi d'une campagne. Une attention particulière a été portée sur la sécurité ainsi qu'au respect du RGPD.

4 Annexes

4.1 Intelligence artificielle

L'intelligence artificielle a été utilisée comme outil d'assistance tout au long du projet, sans remplacer le travail de conception et de développement. Son utilisation s'est principalement concentrée sur trois aspects.

Elle a d'abord contribué à la rédaction du rapport en aidant à reformuler certains passages, clarifier des explications techniques et améliorer la fluidité du texte. Cela a permis de produire un document plus clair, cohérent et structuré, tout en conservant le contenu et les idées du projet.

L'IA a également été utilisée pour apporter un soutien lors de problèmes techniques ponctuels, à la manière d'une ressource documentaire ou d'un forum spécialisé. Les solutions proposées ont toujours été analysées, adaptées et validées avant leur intégration.

Ainsi, l'intelligence artificielle a servi d'outil d'apprentissage et d'optimisation, tout en laissant la réflexion, la conception et la compréhension technique au centre du projet.

4.2 Journal de travail

4.2.1 Total des heures :

Titre	Temps effectué	Planifié
Journal de travail	03:00	10:00
Rapport	39:52	10:00
Recherches RGPD + ajustement (~1h)	01:35	15:00
Planification	02:00	03:00
Maquettes	02:50	08:00
Gestion des rôles	03:45	06:00
Système d'inscription/désinscription	04:25	04:00
Interface de composition basique & Interface de sélection des contact	04:45	08:00
Implémentation de la file d'attente & développement du processus de relais	06:05	08:00
Système de tracking	02:55	06:00
Réalisation des tests	03:35	05:00
Tâches non planifiée	15:13	00:00

4.2.2 Pondération des tâches

Titre	Pourcentage	Pourcentage effectuées
Analyse	20%	6.94%
Implémentation	45%	41.63%
Tests	15%	3.98%
Documentation	20%	47.44%

Pondération des tâches planifiées

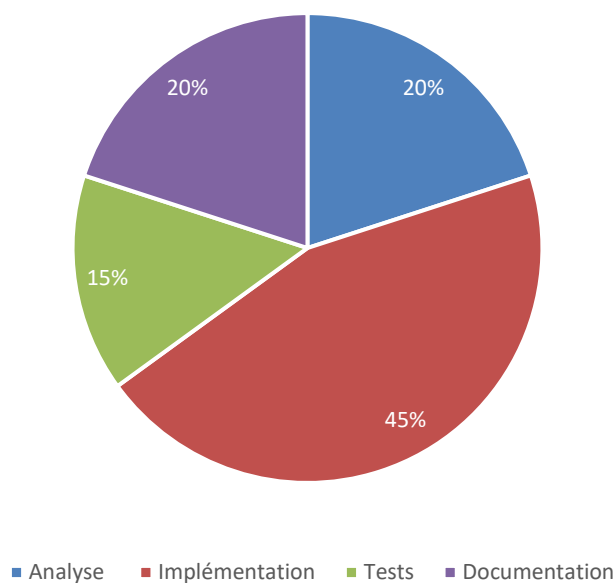


Figure 57. Diagramme circulaire représentant la pondération des tâches planifiées

Pondération des tâches effectuées

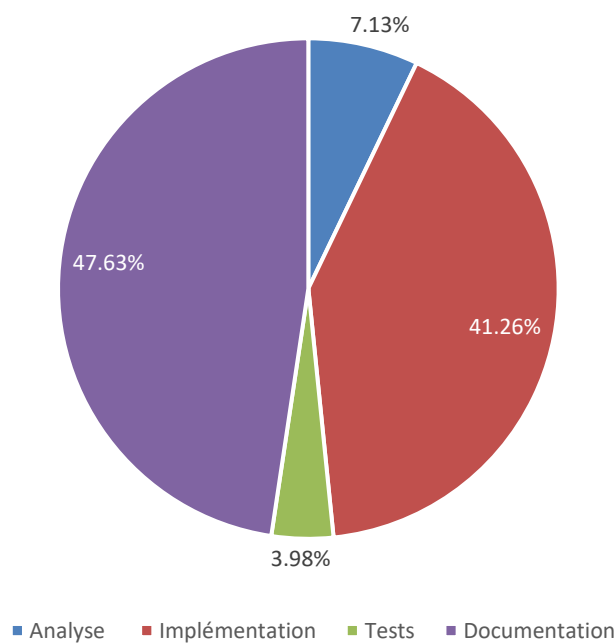


Figure 58. Diagramme circulaire représentant la pondération des tâches effectuées

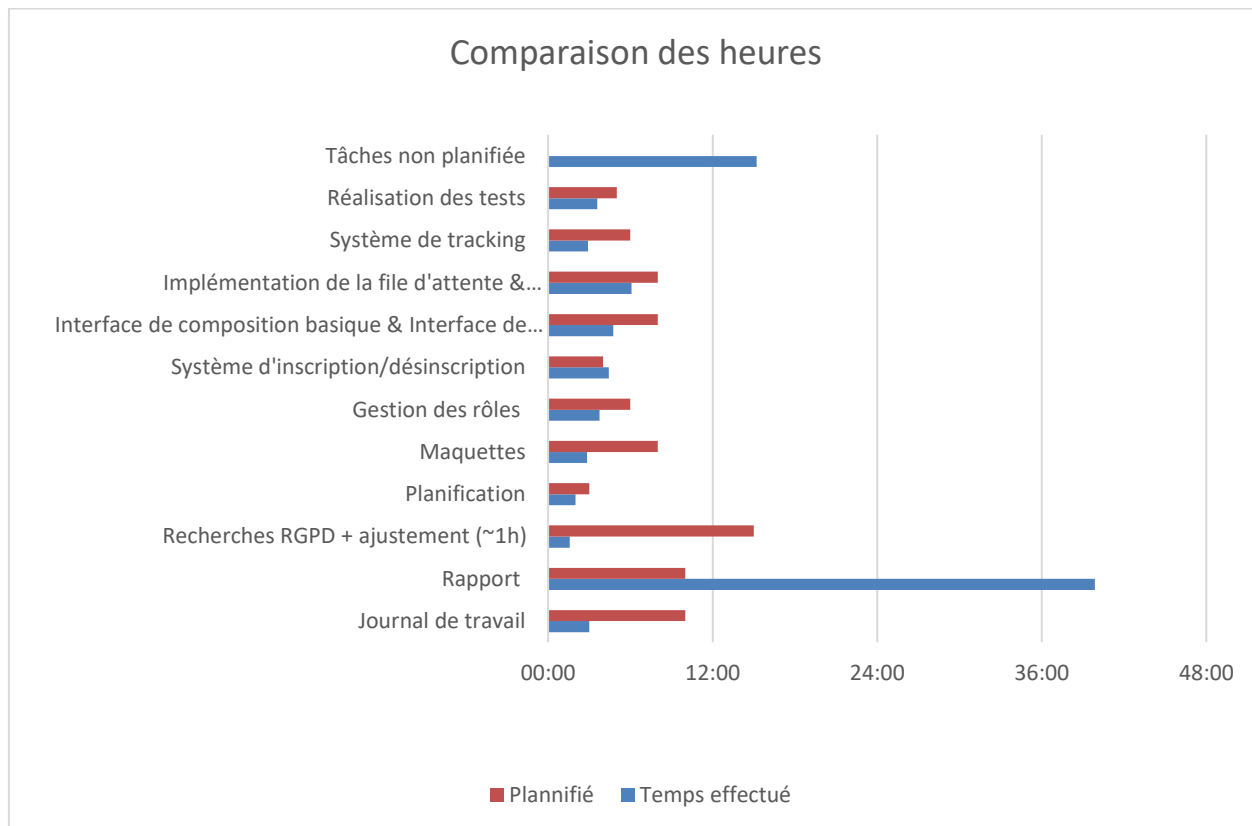


Figure 59. Diagramme en barres pour la comparaison des heures de travail

4.2.3 Explications

La comparaison entre les heures planifiées et les heures effectuées met en évidence plusieurs écarts significatifs qui doivent être expliqués.

Dans un premier temps, le temps consacré au rapport a largement dépassé les prévisions initiales. Cela s'explique principalement par deux raisons :

1. Étant donné que la documentation constitue l'élément le plus important du TPI, il a été décidé de lui accorder une attention particulière afin de produire un contenu de qualité, cohérent et soigné.
2. La charge de travail nécessaire à la réalisation de certaines parties du rapport a été clairement sous-estimée, ce qui a entraîné un investissement en temps plus important que prévu.

Un second point mérite également une explication concernant le journal de travail. Celui-ci a été intégré à la planification comme une tâche distincte, ce qui s'est finalement révélé être une erreur. En effet, cette activité est réalisée progressivement tout au long du projet, à raison de petites mises à jour quotidiennes. Elle aurait donc davantage dû être considérée comme une tâche de fond continue plutôt que comme un élément séparé du planning. Afin de se rapprocher au mieux de la réalité, le temps total consacré à cette tâche a finalement été estimé à environ 3 heures.

Ensuite, il peut être constaté que la majorité des tâches ont nécessité moins de temps que prévu. Cela s'explique notamment par le fait que l'application ne présentait pas de difficultés techniques majeures. De plus, certaines portions de code ainsi que plusieurs maquettes ont pu être réutilisées à partir d'anciens projets, notamment le projet P-APPRO2, ce qui a permis de gagner un temps considérable lors du développement.

Pour finir, plusieurs éléments n'avaient pas été planifiés dans la planification initiale. Tout d'abord, le tableau de bord n'avait tout simplement pas été pris en compte lors de la planification alors qu'il représente un élément central de l'application. La planification de deux autres éléments, notamment la page de statistiques et la mise en production, n'a pas été effectuée car ils ne faisaient pas partie des demandes initiales du projet. Veuillez noter que les différents échanges avec les experts ou les chefs de projet n'ont également pas été intégrés à la planification.

4.2.4 Journal

Le journal de travail se trouve dans l'annexe "[Journal de travail](#)"

4.3 Sources et références

Recherches sur le RGPD :

- <https://eur-lex.europa.eu/legal-content/FR/TXT/?uri=celex:32002L0058>
- <https://www.privacy-regulation.eu/fr/>
- <https://analytics-docs.piano.io/fr/analytics/v1/consent-exemption-configuration>

Analyse DNS :

- <https://www.hardenize.com/report/section-inf.ch/1778663193#summary>
- <https://mha.azurewebsites.net/>
- <https://www.cloudflare.com/fr-fr/learning/dns/dns-records/dns-dmarc-record/>
- <https://www.cloudflare.com/fr-fr/learning/dns/dns-records/dns-spf-record/>

Documentation Next.js :

- <https://nextjs.org/docs>

Documentation Tailwind :

- <https://v2.tailwindcss.com/docs>

Documentation Prisma ORM :

- <https://www.prisma.io/docs>

4.4 Liste des livrables remis

Livable	Emplacement (lien/nom du fichier)
Code source	https://github.com/ThomNardou/Agora
Journal de travail	https://github.com/ThomNardou/Agora/blob/main/Doc/ThomNardou-JDT.xlsx
Rapport de projet	https://github.com/ThomNardou/Agora/blob/main/Doc/TPI_Rapport_ThomNardou.docx
Staging	https://agora.w3.pm2etml.ch/
Vidéo	https://youtu.be/OVMvhF9IYZU
Journal de travail	ThomNardou-JDT.pdf
Planification initiale	Plannification\ThoNardou-Agora-Plannification_V2.pdf
Maquettes BF	Maquettes\Maquettes_basse.pdf
Maquettes HF	Maquettes\Maquettes_haute.pdf
Release	https://github.com/ThomNardou/Agora/releases/tag/1.0.0